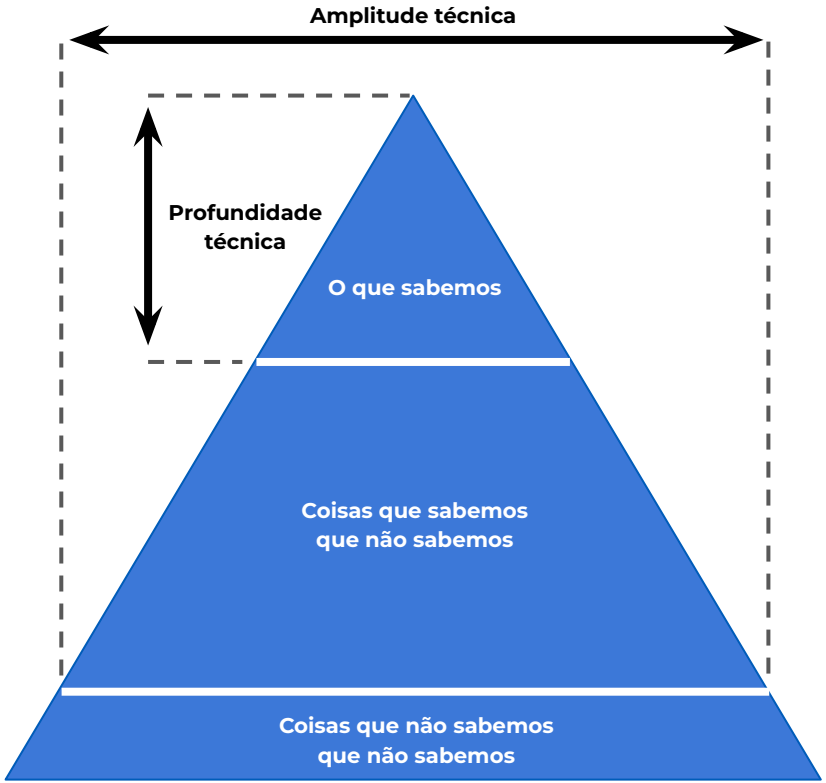
CLUBE DE ESTUDOS
BY ELEMAR JR

C#
DO JEITO CERTO

O QUE NÃO SABEMOS?



AULA 1

DOMINANDO O
GERENCIAMENTO DE MEMÓRIA

QUAL O PROBLEMA ?

ABORDAGEM 1

```
var sql = @"SELECT Id, Name, Age
            FROM Person
            WHERE <<conditions[0]>> AND <<conditions[1]>> AND <<conditions[2]>>";

var index = 0;
foreach (var condition in conditions)
{
    sql = sql.Replace($"<<conditions[{index++}]>>", condition);
}

using var connection = new NpgsqlConnection(connectionString);
var results = await connection.QueryAsync<PersonDto>(sql, parameters);
```

ABORDAGEM 2

```
var sql = @"SELECT Id, Name, Age
            FROM Person
            WHERE {0} AND {1} AND {2}";

sql = string.Format(sql, conditions);

using var connection = new NpgsqlConnection(connectionString);
var results = await connection.QueryAsync<PersonDto>(sql, parameters);
```

QUAL A DIFERENÇA ?

Method	Mean	Error	StdDev	Gen0	Allocated
Replace	129.43 ns	2.621 ns	7.218 ns	0.0629	1056 B
Format	74.59 ns	1.546 ns	2.496 ns	0.0200	336 B

EXEMPLO REAL

5 PILARES IMPORTANTE PARA PENSARMOS

Reference and Value Type

Heap and Stack

Box and Unboxing

Class, Record and Struct

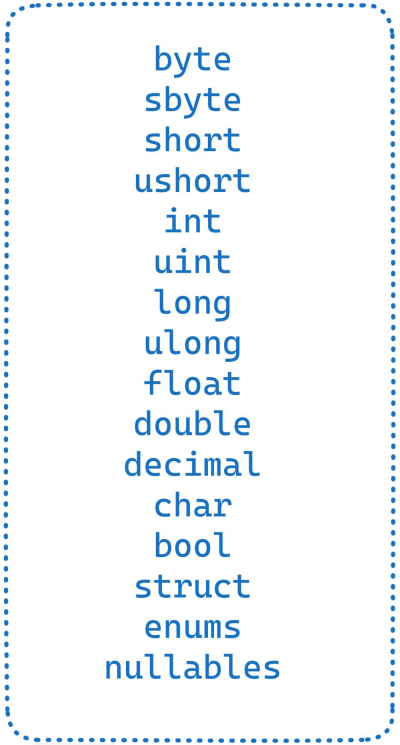
Garbage Collector

CAPÍTULO 1

VALUE AND REFERENCE TYPE.

VALUE TYPES

SÃO TIPOS QUE **CARREGAM O VALOR “DENTRO” DA VARIÁVEL**, OU SEJA, NÃO NECESSITA GUARDAR UM ENDEREÇO DE REFERÊNCIA EM OUTRO LUGAR DA MEMÓRIA.

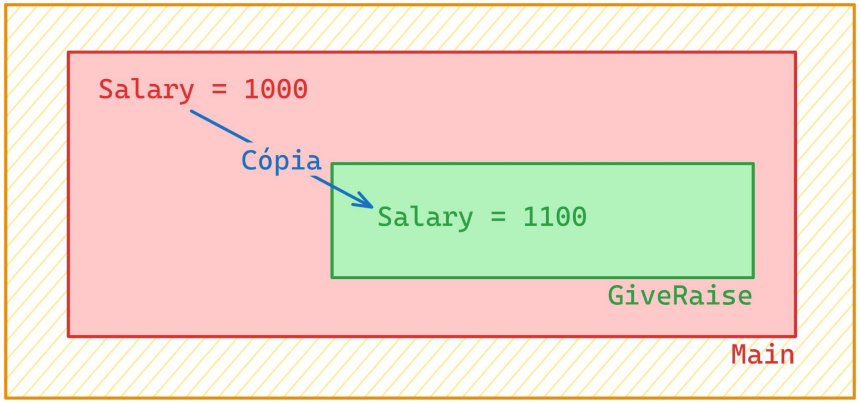


TODOS OS TIPOS PRIMITIVOS SÃO VALUE TYPE

```
var salary = 1000d;

GiveRaise(salary);

Console.WriteLine($"The new salary is: {salary}");

void GiveRaise(double salary)
{
    salary = salary * 1.10;
}
```

EXEMPLO

```
var salary = 1000d;

GiveRaise(salary);

Console.WriteLine($"The new salary is: {salary}");

void GiveRaise(double salary)
{
    salary = salary * 1.10;
}
```




```
"The new salary is: 1000"
```

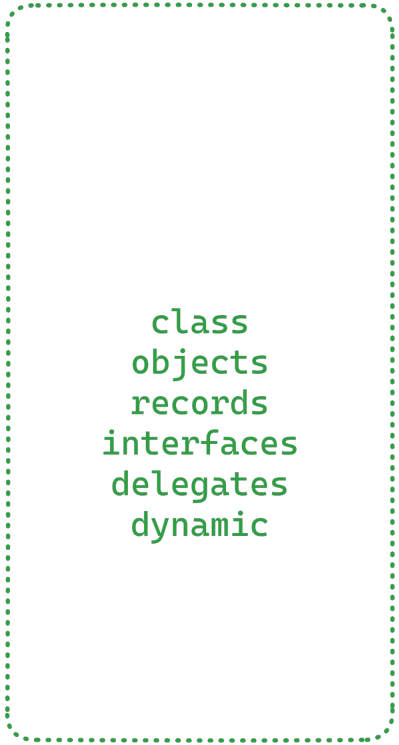
```
var salary = 1000d;

GiveRaise(ref salary);

Console.WriteLine($"The new salary is: {salary}");

void GiveRaise(ref double salary)
{
    salary = salary * 1.10;
}
```

```
var salary = 1000d;

GiveRaise(ref salary);

Console.WriteLine($"The new salary is: {salary}");

void GiveRaise(ref double salary)
{
    var newSalary = salary;
    newSalary *= 1.10;
}
```

```
var firstPoint = new Point(x: 10, y: 10);
var secondPoint = new Point(x: 10, y: 10);

Console.WriteLine(firstPoint.Equals(secondPoint));


public struct Point
{
    public double X { get; private set; }
    public double Y { get; private set; }

    public Point(double x, double y)
    {
        X = x;
        Y = y;
    }
}
```

REFERENCE TYPE

SÃO TIPOS QUE **GUARDAM APENAS O ENDEREÇO (REFERÊNCIA)** DE ONDE O DADO ESTÁ LOCALIZADO. OU SEJA, O VALOR REAL VIVE EM OUTRO LUGAR E A **VARIÁVEL FUNCIONA COMO UM PONTEIRO** PARA CHEGAR LÁ.

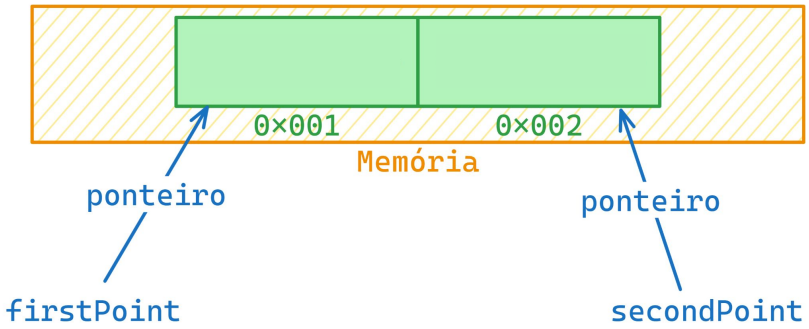


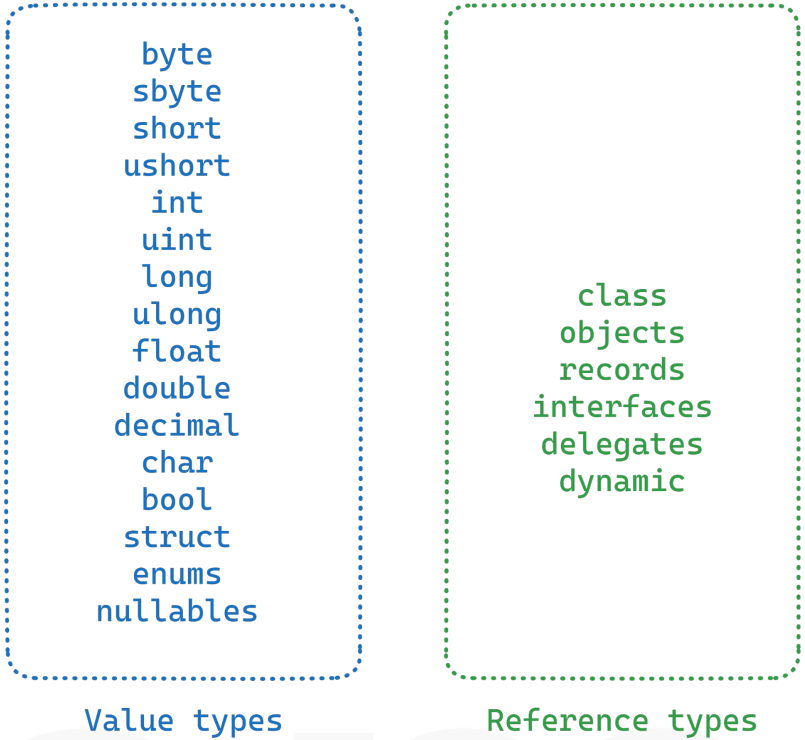
Reference types

TIPOS COMPLEXOS QUE FUNCIONAM COMO **PONTEIROS DE MEMÓRIA**

EXEMPLO

```
var firstPoint = new Point(x: 10, y: 10);
var secondPoint = new Point(x: 10, y: 10);

Console.WriteLine(firstPoint.Equals(secondPoint));

public class Point
{
    public double X { get; private set; }
    public double Y { get; private set; }

    public Point(double x, double y)
    {
        X = x;
        Y = y;
    }
}
```





STRING: O GRANDE IMPOSTOR

EMBORA **PAREÇA UM TIPO PRIMITIVO** E TENHA SEMÂNTICA DE VALOR (COMPARAÇÃO POR CONTEÚDO, NÃO REFERÊNCIA), A **STRING É UM REFERENCE TYPE**.

```
var tableProduct = "Table";
var tableProduct2 = "Table";

Console.WriteLine(tableProduct == tableProduct2);
Console.WriteLine(tableProduct.Equals(tableProduct2));
```

Código fonte do .NET

```
public static bool operator ==(string? a, string? b) => Equals(a, b);
```

Fonte:https://source.dot.net/#System.Private.CoreLib/src/libraries/System.Private.CoreLib/src/System/String.Comparison.cs,372d790ddae4cbb4

QUAIS DIFERENÇAS ENTRE VALUE E REFERENCE TYPES

VALUE TYPE	REFERENCE TYPE
Guardam o valor	Guardam um ponteiro
Alocados na Stack	Alocados na Heap
Equals compara valor	Equals não compara valor

CAPÍTULO 2

TIPOS DE MEMÓRIA

TIPOS DE MEMÓRIAS: STACK

GUARDA **DADOS DE VIDA CURTA**, LIGADOS À EXECUÇÃO DO MÉTODO (VARIÁVEIS LOCAIS E INFORMAÇÕES DE CHAMADA).

ELA É **RÁPIDA E PREVISÍVEL**: QUANDO O **MÉTODO** TERMINA, ESSE ESPAÇO É LIBERADO PRATICAMENTE “**AUTOMATICAMENTE**”.

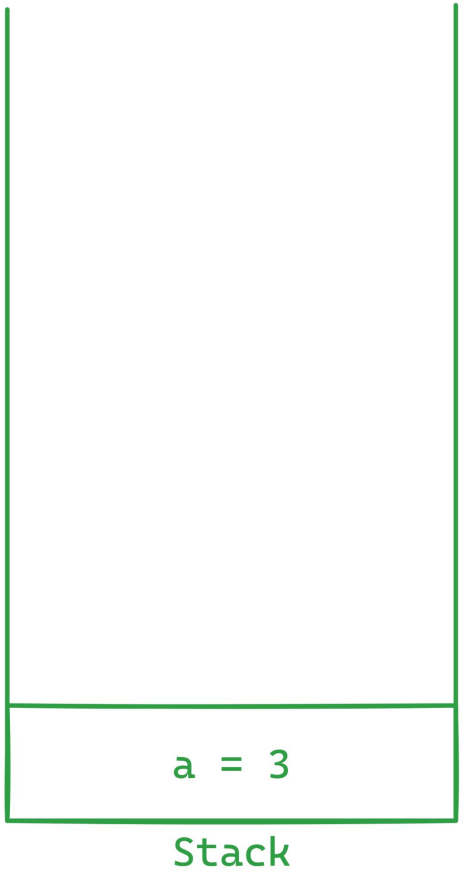
PRINCIPAIS CARACTERÍSTICAS DA STACK

LIMITE DE **1MB PARA 32 bits** / **4MB PARA 64 bits**

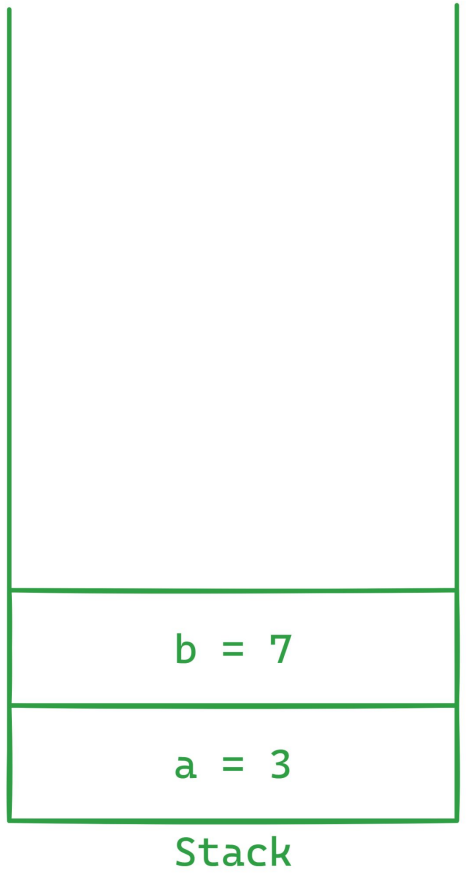
CADA **THREAD TEM SUA STACK**

ARMAZENA **VALUE TYPES E REFERÊNCIAS** PARA REFERENCE TYPES

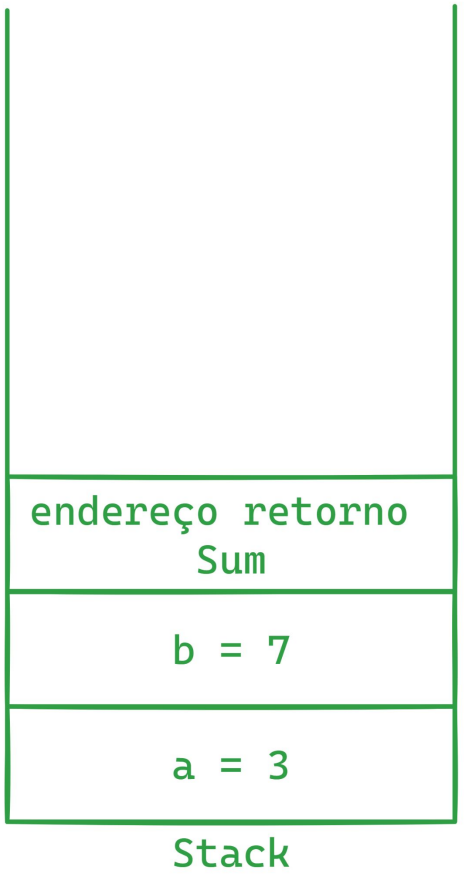
```
var a = 3;
var b = 7;

var result = Sum(a, b);

Console.WriteLine(result);

public int Sum(int a, int b)
{
    return a + b;
}
```



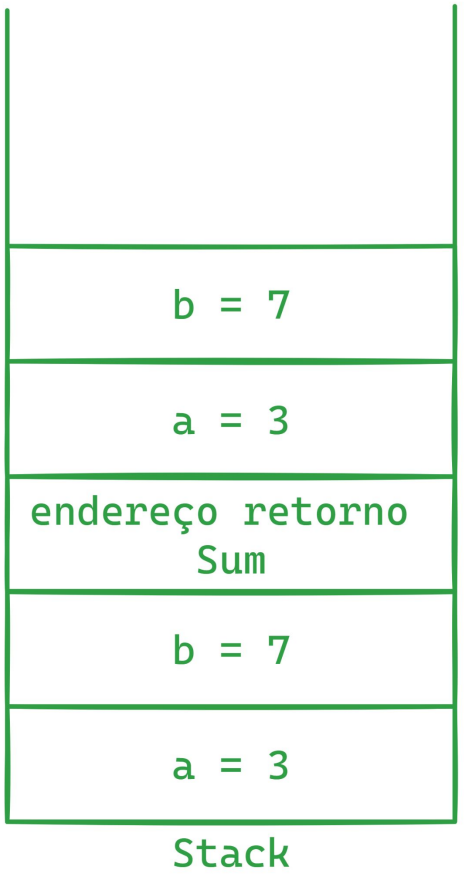
```
var a = 3;
var b = 7;

var result = Sum(a, b);

Console.WriteLine(result);

public int Sum(int a, int b)
{
    return a + b;
}
```



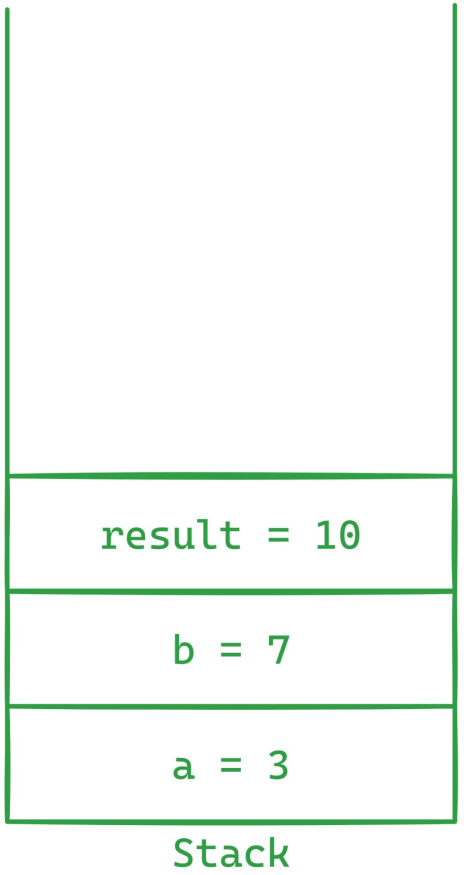
```
var a = 3;
var b = 7;

var result = Sum(a, b);

Console.WriteLine(result);

public int Sum(int a, int b)
{
    return a + b;
}
```



```
var a = 3;
var b = 7;

var result = Sum(a, b);

Console.WriteLine(result);

public int Sum(int a, int b)
{
    return a + b;
}
```



```
var a = 3;
var b = 7;

var result = Sum(a, b);

Console.WriteLine(result);

public int Sum(int a, int b)
{
    return a + b;
}
```



TIPOS DE MEMÓRIAS: HEAP

GUARDA OBJETOS DE **VIDA MAIS LONGA** (NORMALMENTE INSTÂNCIAS DE CLASSES E COISAS QUE PRECISAM **CONTINUAR EXISTINDO FORA DO MÉTODO**).

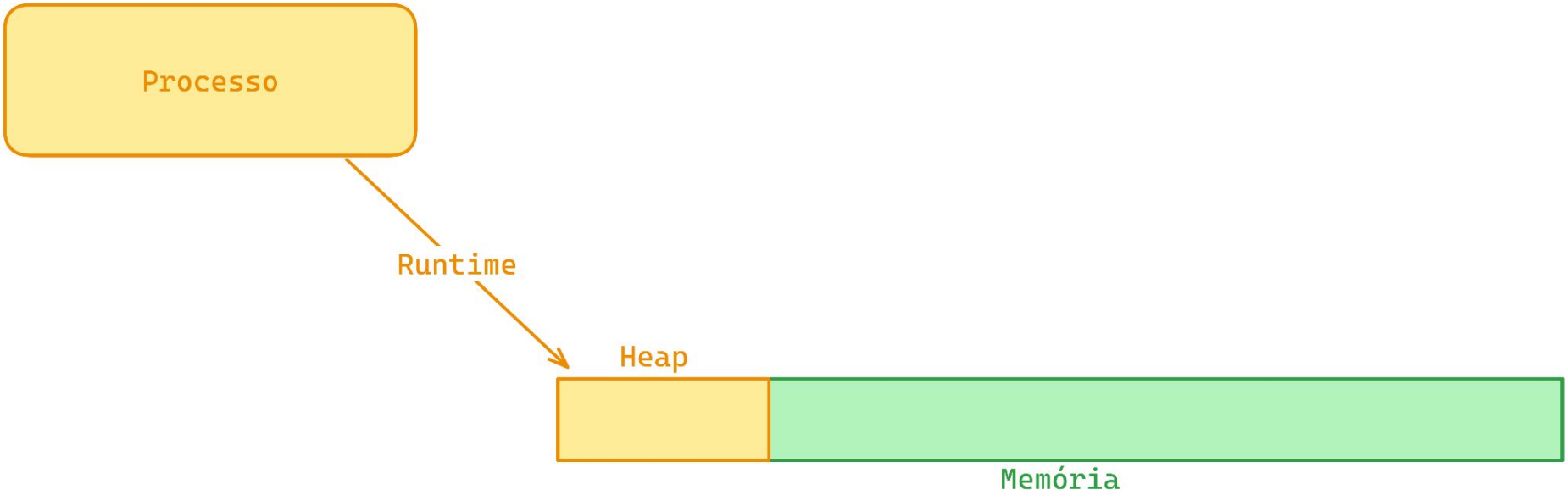
ELE É FLEXÍVEL, MAS **TEM UM CUSTO**: ESSES OBJETOS PRECISAM SER RASTREADOS E LIMPOS PELO **GARBAGE COLLECTOR (GC)**.

PRINCIPAIS CARACTERÍSTICAS DA HEAP

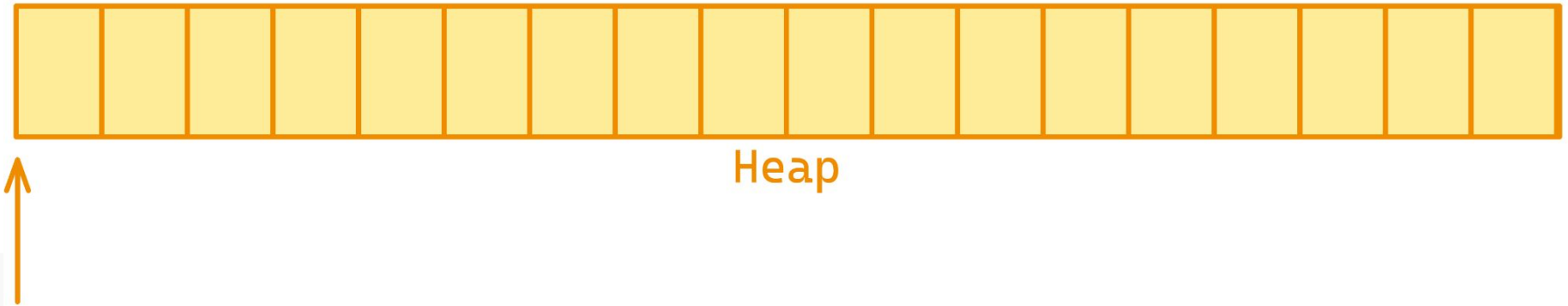
SEM LIMITE DE TAMANHO "FIXO"

PRECISA DO **GARBAGE COLLECTOR PARA LIBERAR ESPAÇO**

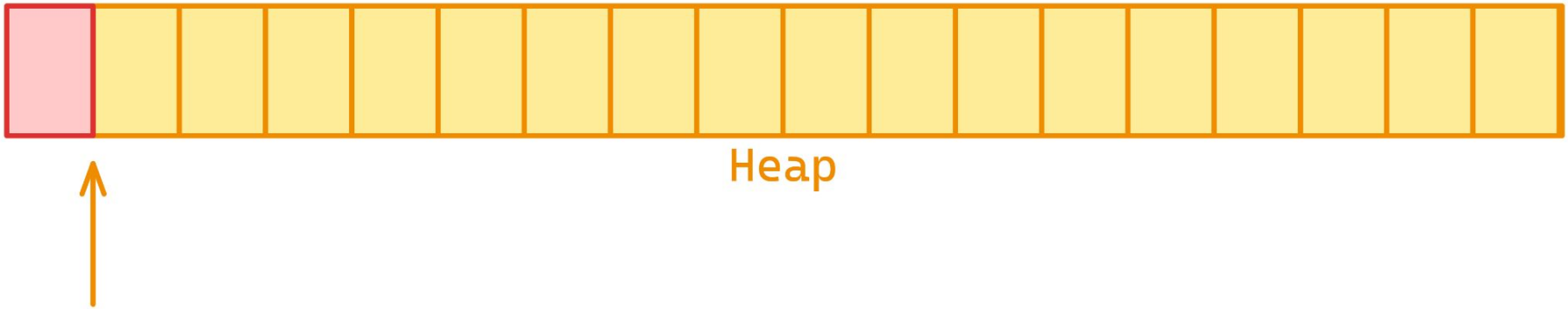
TODAS AS THREADS COMPARTILHAM A HEAP



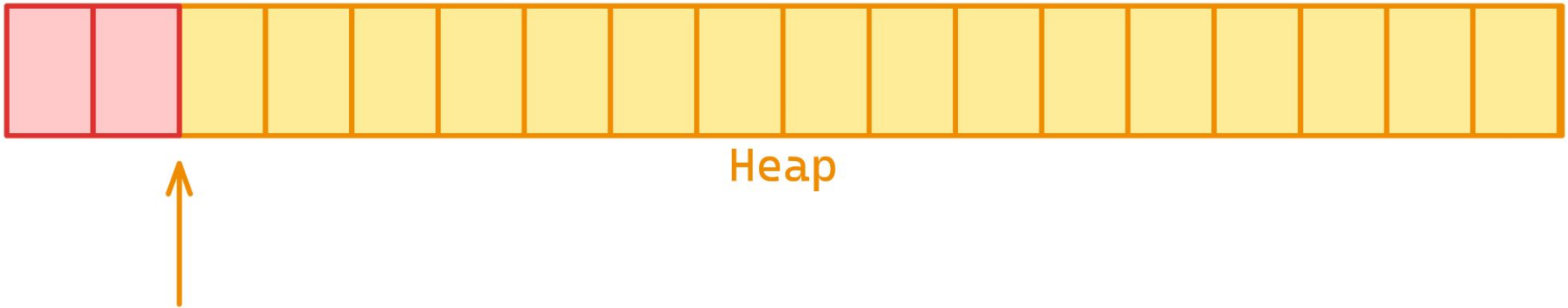
```
var smith = new Person { Name = "Smith"};
var joe = new Person { Name = "Joe"};
var carol = new Person { Name = "Carol"};
```



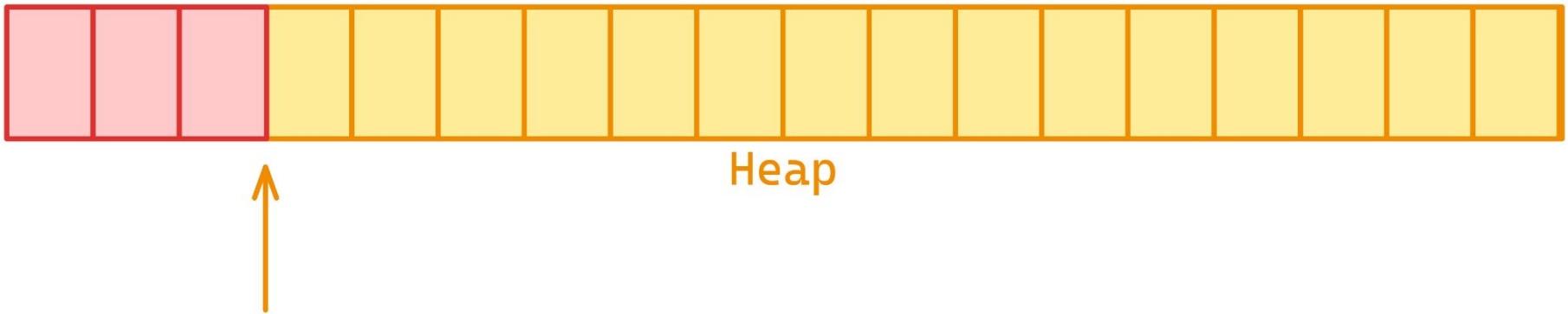
```
var smith = new Person { Name = "Smith"};
var joe = new Person { Name = "Joe"};
var carol = new Person { Name = "Carol"};
```



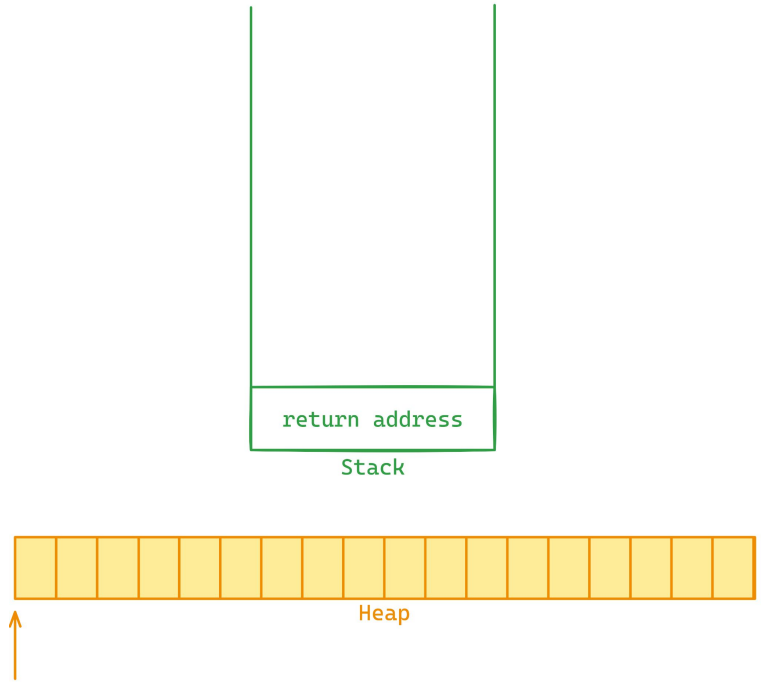
```
var smith = new Person { Name = "Smith"};
var joe = new Person { Name = "Joe"};
var carol = new Person { Name = "Carol"};
```



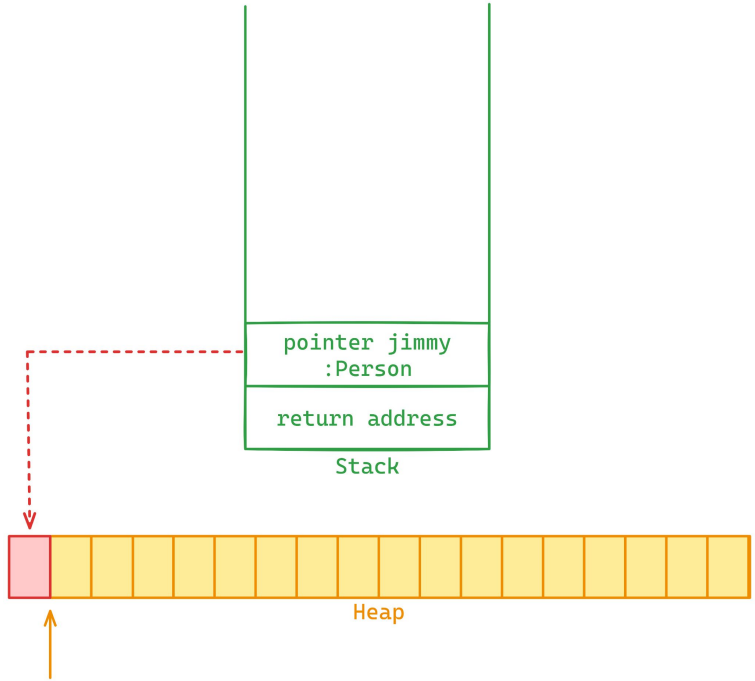
```
var smith = new Person { Name = "Smith"};
var joe = new Person { Name = "Joe"};
var carol = new Person { Name = "Carol"};
```



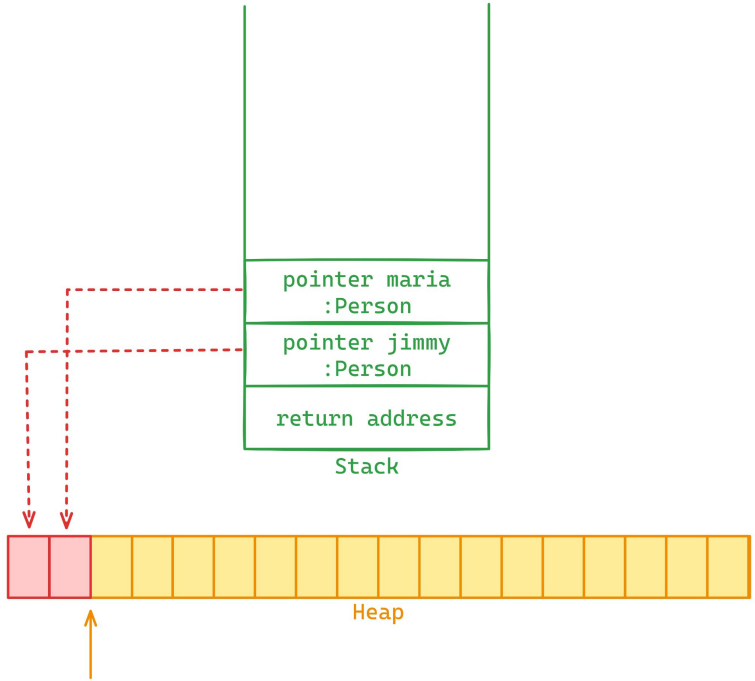
```
var jimmy = CreateSomePeople();

Console.WriteLine(jimmy.Age);

Person CreateSomePeople()
{
    var jimmy = new Person("Jimmy", age: 33);
    var maria = new Person("Maria", age: 33);
    var smith = new Person("Smith", age: 33);

    return jimmy;
}
```



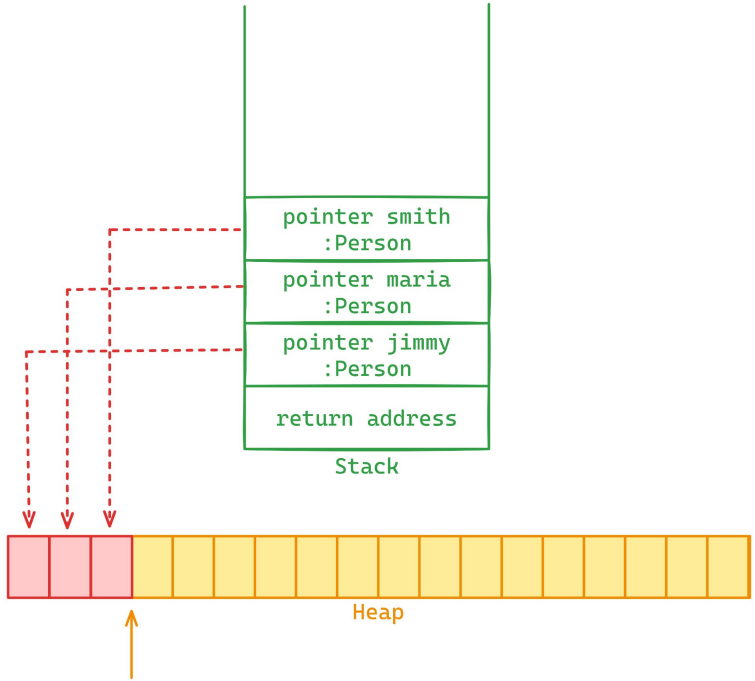
```
var jimmy = CreateSomePeople();

Console.WriteLine(jimmy.Age);

Person CreateSomePeople()
{
    var jimmy = new Person("Jimmy", age: 33);
    var maria = new Person("Maria", age: 33);
    var smith = new Person("Smith", age: 33);

    return jimmy;
}
```



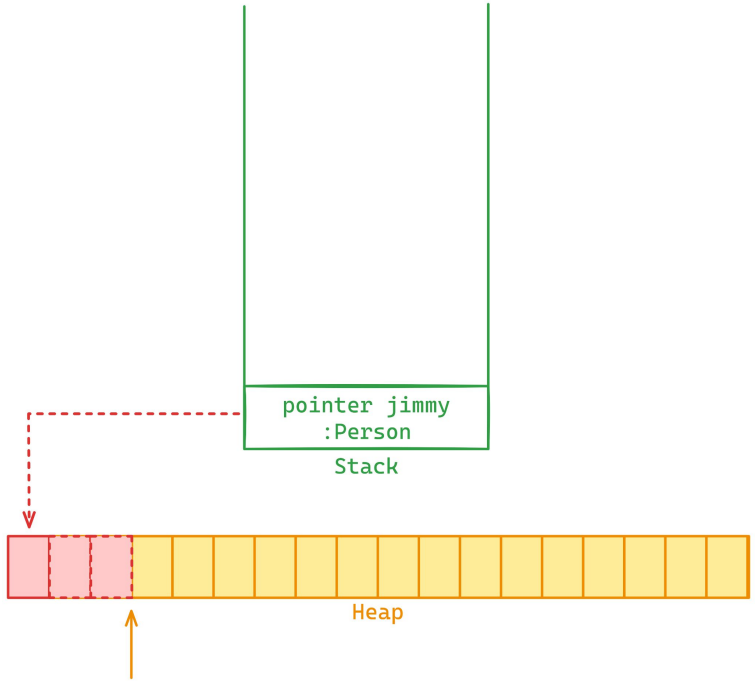
```
var jimmy = CreateSomePeople();

Console.WriteLine(jimmy.Age);

Person CreateSomePeople()
{
    var jimmy = new Person("Jimmy", age: 33);
    var maria = new Person("Maria", age: 33);
    var smith = new Person("Smith", age: 33);

    return jimmy;
}
```



```
var jimmy = CreateSomePeople();

Console.WriteLine(jimmy.Age);

Person CreateSomePeople()
{
    var jimmy = new Person("Jimmy", age: 33);
    var maria = new Person("Maria", age: 33);
    var smith = new Person("Smith", age: 33);

    return jimmy;
}
```



```
var jimmy = CreateSomePeople();

Console.WriteLine(jimmy.Age);

Person CreateSomePeople()
{
    var jimmy = new Person("Jimmy", age: 33);
    var maria = new Person("Maria", age: 33);
    var smith = new Person("Smith", age: 33);

    return jimmy;
}
```



BOXING / UNBOXING

UM VILÃO **INVISÍVEL**

BOXING/UNBOXING: O “IMPOSTO INVISÍVEL”

VOCÊ ESCREVE ALGO QUE **PARECE UMA ATRIBUIÇÃO/CONVERSÃO SIMPLES**, MAS POR BAIXO ACONTECE ALOCAÇÃO NO HEAP E ISSO **VIRA PRESSÃO NO GC E CPU**.

PRINCIPAIS DIFERENÇAS DO BOXING E UNBOXING

BOXING CONVERTE UM **VALUE TYPE** PARA UM **REFERENCE TYPE**

UNBOXING CONVERTE UM **REFERENCE TYPE** PARA UM **VALUE TYPE.**

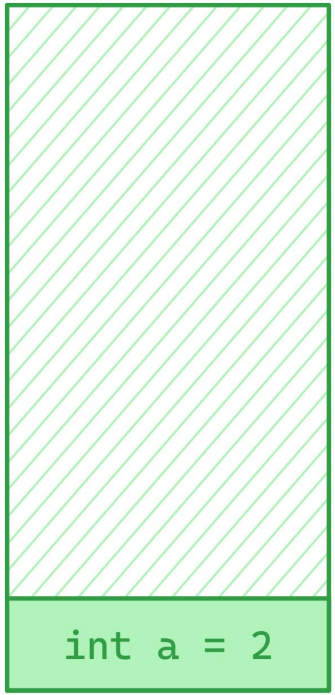
OPERAÇÃO RELATIVAMENTE **CUSTOSA**

BOXING ALOCA UM NOVO OBJETO NA **HEAP**

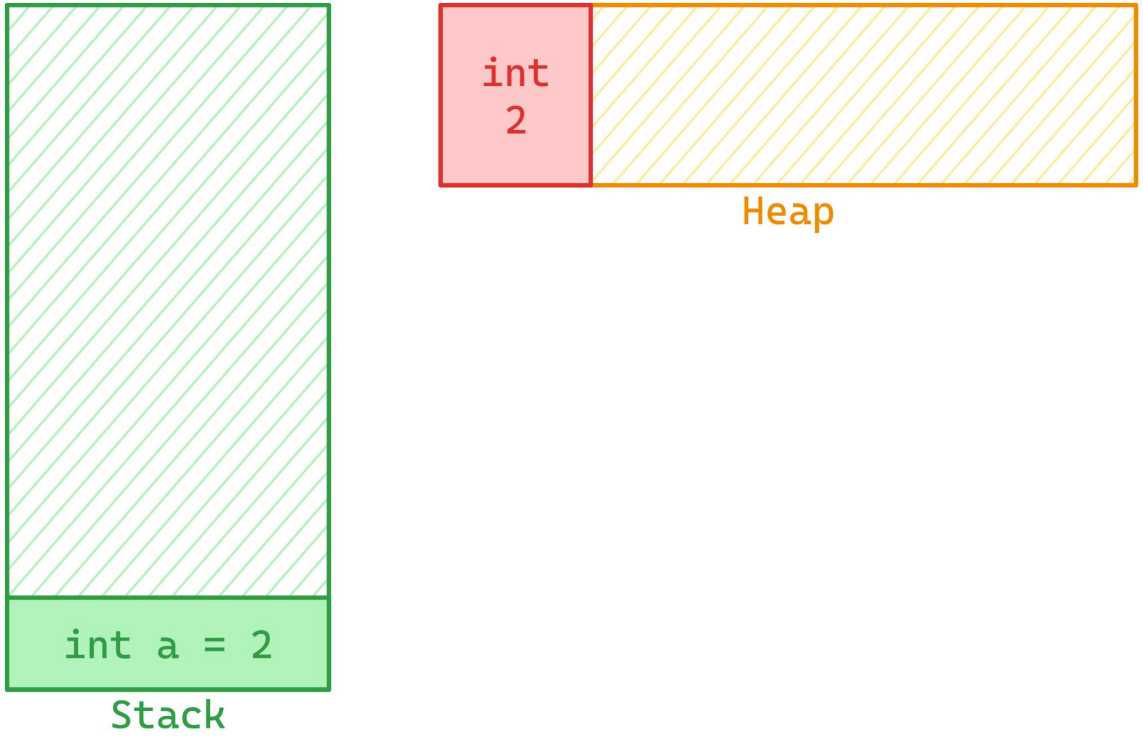
```
public string Concat() =>
    string.Concat("The final salary is: ", 1000);

public string Interpolation() =>
    $"The final salary is: {1000}";

public string StringBuilder() =>
    _sb.Append("The final salary is: ").Append(1000).ToString();
```

Method	Mean	Error	StdDev	Gen0	Allocated
Concat	16.41 ns	0.412 ns	1.182 ns	0.0076	128 B
Interpolation	36.39 ns	0.742 ns	0.825 ns	0.0043	72 B
StringBuilder	13.88 ns	0.304 ns	0.509 ns	0.0043	72 B

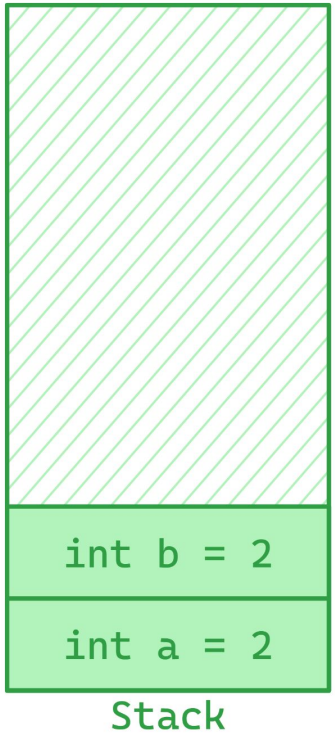
```
int a = 2;
object boxed = a;
int b = (int)boxed;
```



```
int a = 2;
object boxed = a;
int b = (int)boxed;
```



```
int a = 2;
object boxed = a;
int b = (int)boxed;
```

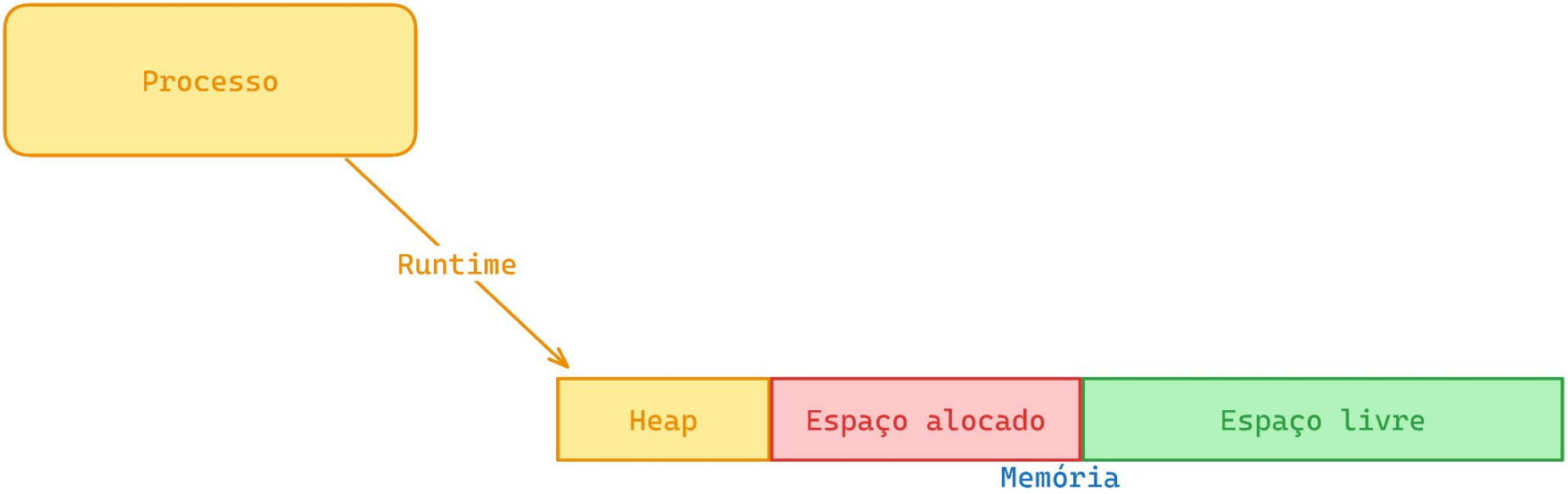


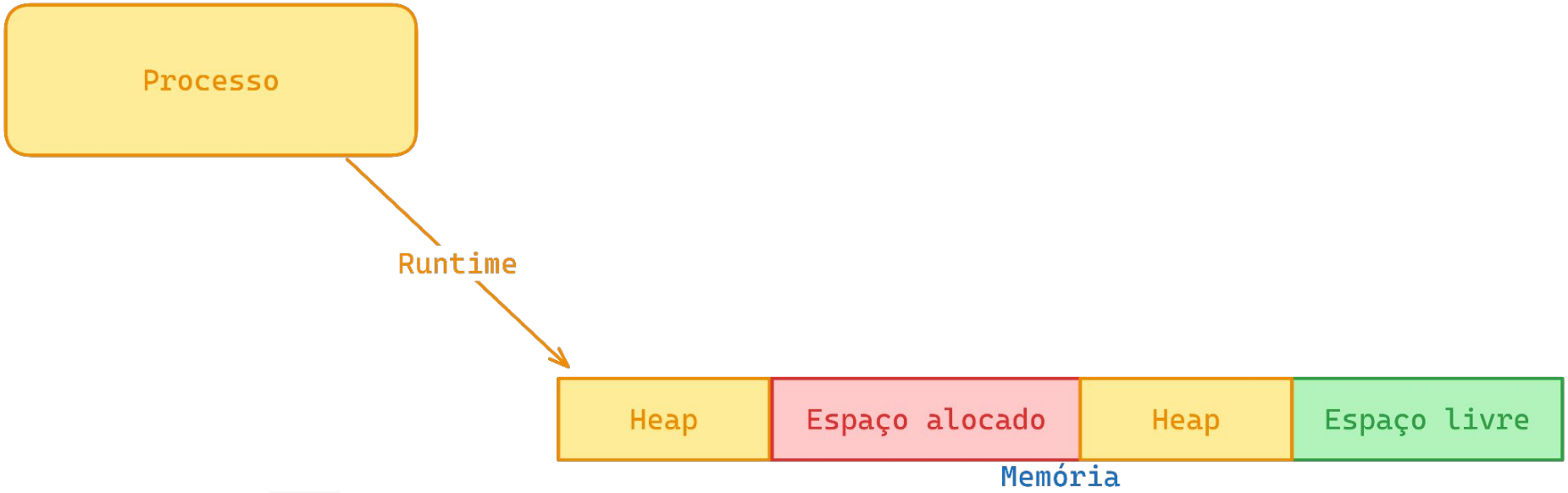
```
public struct Point
{
    ....
    public override bool Equals(object? obj)
    {
        if (obj is not Point point) return false;
        return point.X == X && point.Y == Y;
    }

    public bool Equals(Point obj)
    {
       return obj.X == X && obj.Y == Y;
    }
}
```

Method	Mean	Error	StdDev	Gen0	Allocated
WithBoxing	2.8802 ns	0.9182 ns	0.0503 ns	0.0014	24 B
WithoutBoxing	0.1737 ns	0.0376 ns	0.0021 ns	-	-

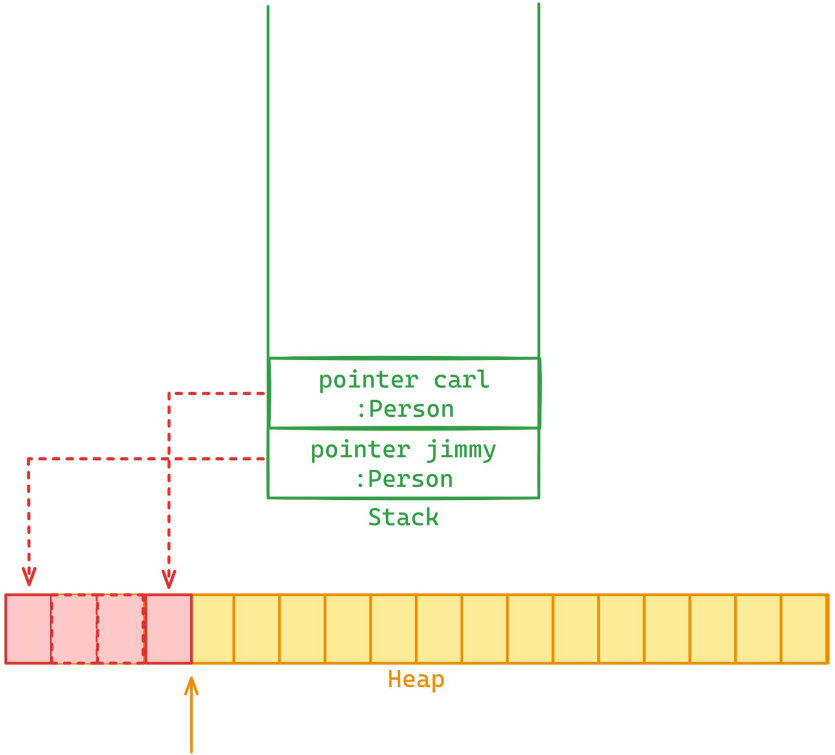
TIPOS DE MEMÓRIAS: HEAP OBJETOS MAIORES COMO CLASSES E ARRAYS

E SE EXCEDERMOS O TAMANHO DA **HEAP** ?





```
var jimmy = CreateSomePeople();
var carl = new Person("Carl", age: 34);

Console.WriteLine(jimmy.Age);
Console.WriteLine(carl.Age);

Person CreateSomePeople()
{
    var jimmy = new Person("Jimmy", age: 33);
    var maria = new Person("Maria", age: 33);
    var smith = new Person("Smith", age: 33);

    return jimmy;
}
```



CAPÍTULO 3

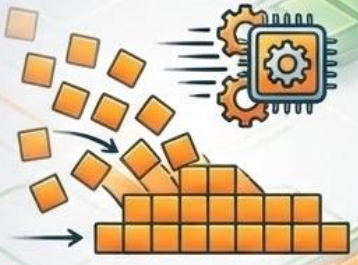
GARBAGE COLLECTOR

GARBAGE COLLECTOR

O **GARBAGE COLLECTOR (GC)** NO .NET É O COMPONENTE RESPONSÁVEL POR GERENCIAR AUTOMATICAMENTE A MEMÓRIA DOS OBJETOS ALOCADOS NO **MANAGED HEAP**.

GARBAGE COLLECTOR

Libera espaço na heap removendo objetos alocados que não possuem referência



Quanto mais alocação na heap, mais o GC trabalha

Elimina a necessidade de liberação de memória de forma manual (managed resources).

Dividido em 3 gerações: Gen0, Gen1, Gen2

GARBAGE COLLECTOR

CONDIÇÕES PARA ACIONAMENTO

O SISTEMA DETECTA BAIXA MEMÓRIA DISPONÍVEL

Gatilho automático quando a memória do sistema está crítica, antes de um erro OutOfMemory.

OBJETOS ALOCADOS NA HEAP ULTRAPASSAM O LIMIAR ESTABELECIDO

O GC atua quando a heap atinge um tamanho predefinido para a geração (Gen0, Gen1, etc.).

MANUALMENTE DISPARADO ATRAVÉS DO GC.COLLECT (EVITE FAZER ISSO)

Intervenção manual, geralmente não recomendada pois pode prejudicar a performance.

GARBAGE COLLECTOR

ETAPAS DE FUNCIONAMENTO

1. SUSPENSÃO

Suspende toda a aplicação para evitar problemas de concorrência no acesso a memória.

2. MARCAÇÃO

Analisa a heap e "marca" objetos que ainda possuem referência.

3. LIMPEZA

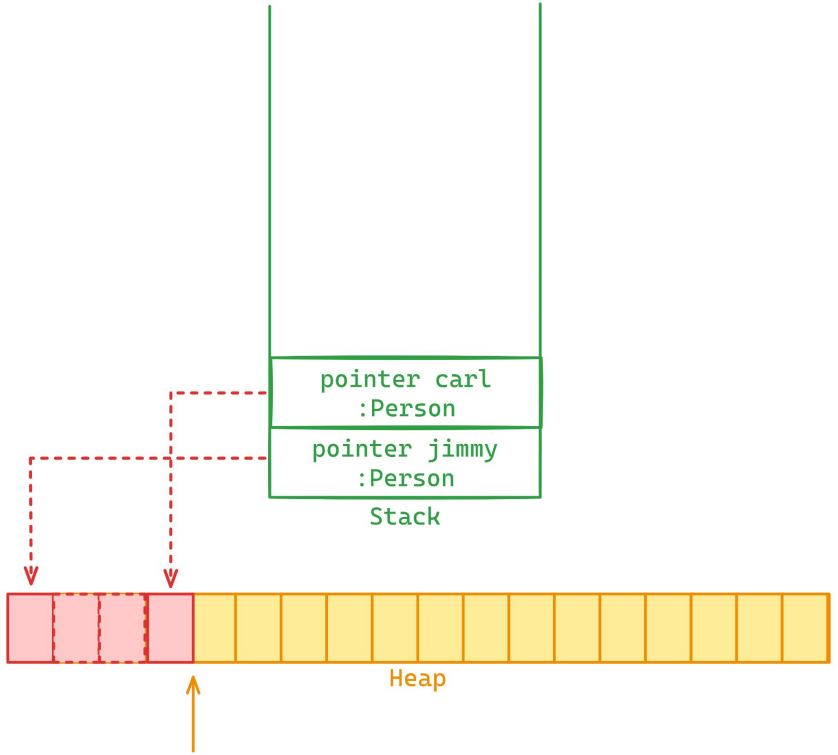
Objetos que não foram marcados na etapa anterior são removidos da memória.

4. COMPACTAÇÃO

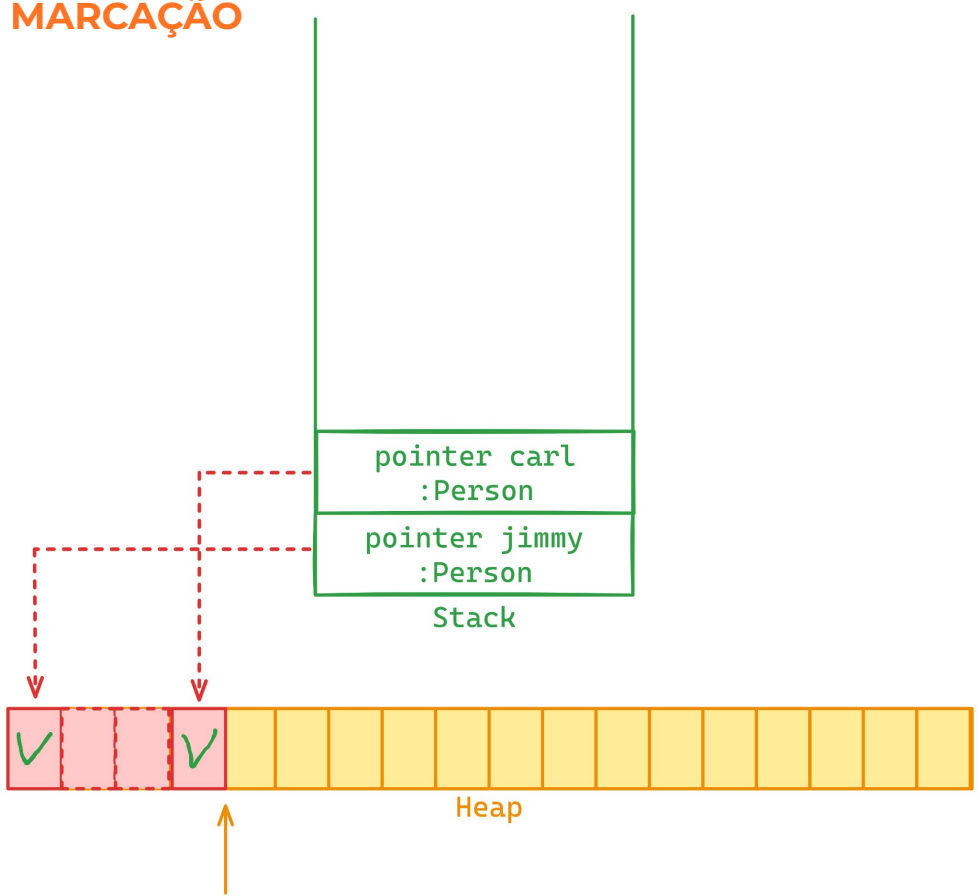
Espaços liberados na etapa anterior são organizados/ compactados.

GARBAGE COLLECTOR: SUSPENSÃO

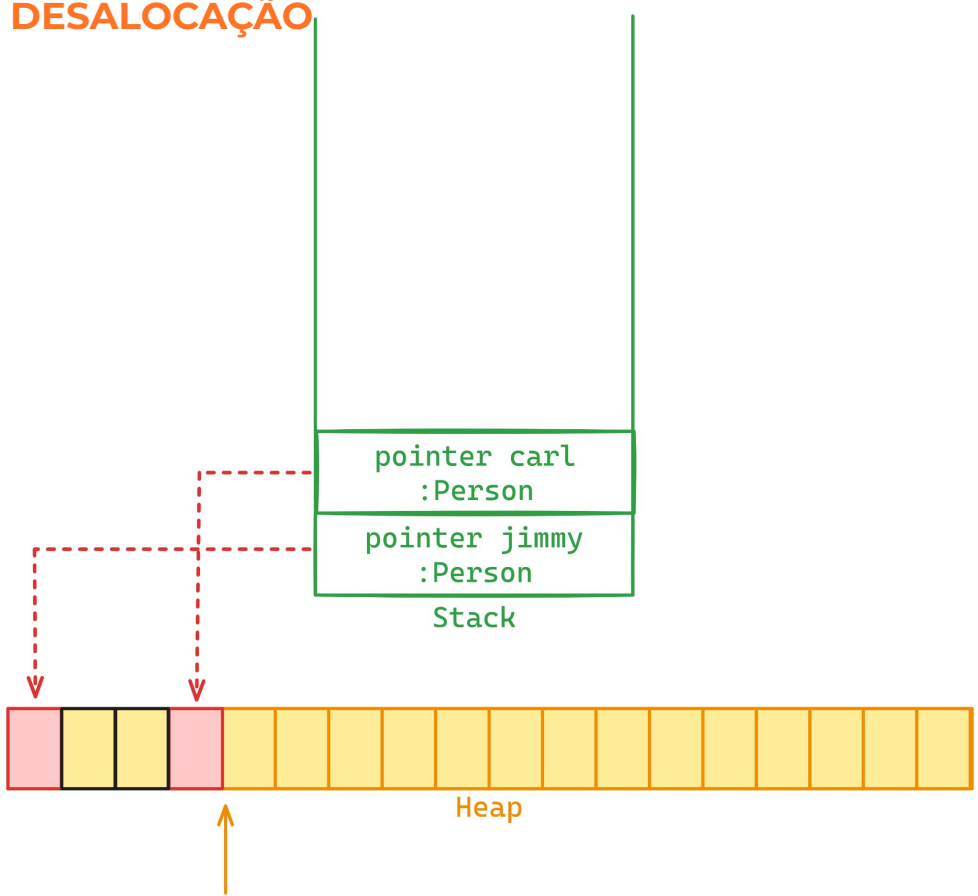
```
var jimmy = CreateSomePeople();
var carl = new Person("Carl", age: 34);

Console.WriteLine(jimmy.Age);
Console.WriteLine(carl.Age);

Person CreateSomePeople()
{
    var jimmy = new Person("Jimmy", age: 33);
    var maria = new Person("Maria", age: 33);
    var smith = new Person("Smith", age: 33);

    return jimmy;
}
```



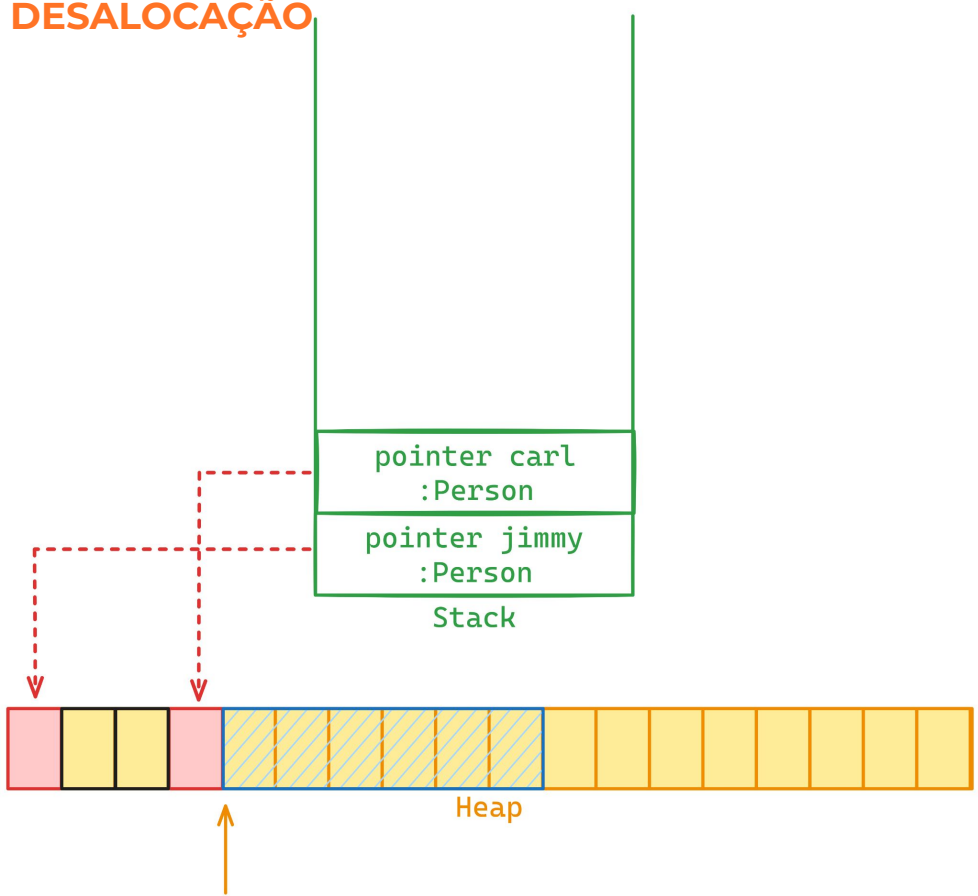
GARBAGE COLLECTOR: MARCAÇÃO



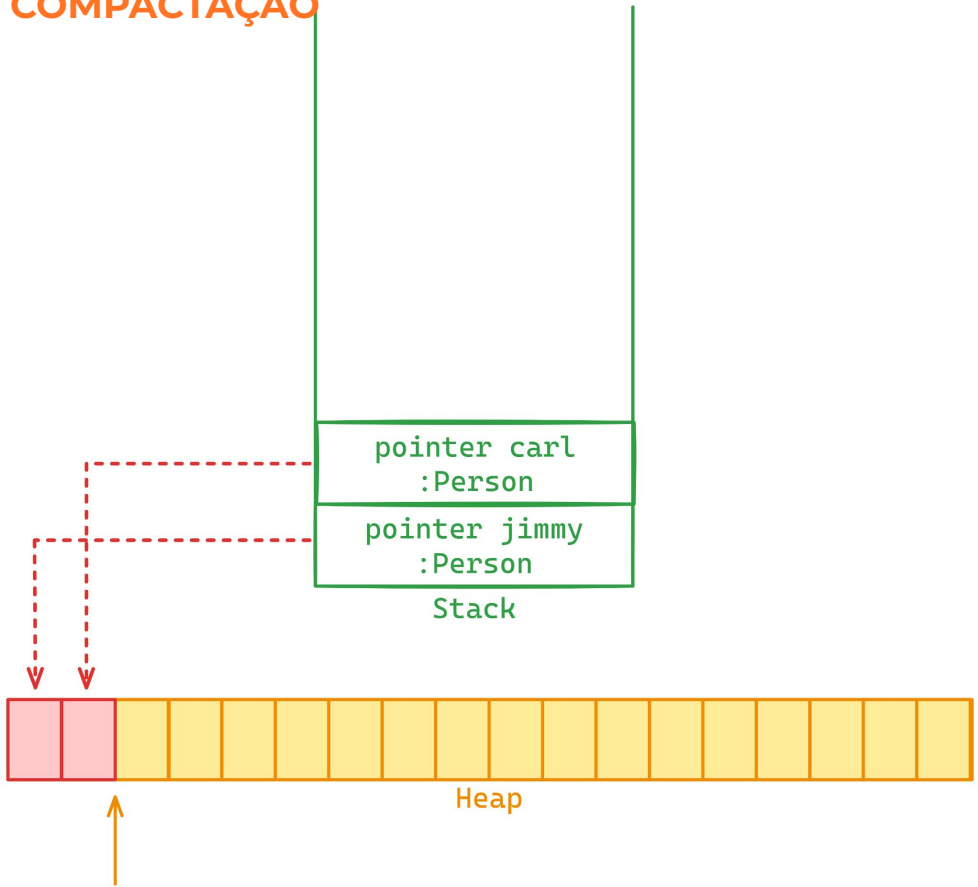
GARBAGE COLLECTOR: DESALOCAÇÃO



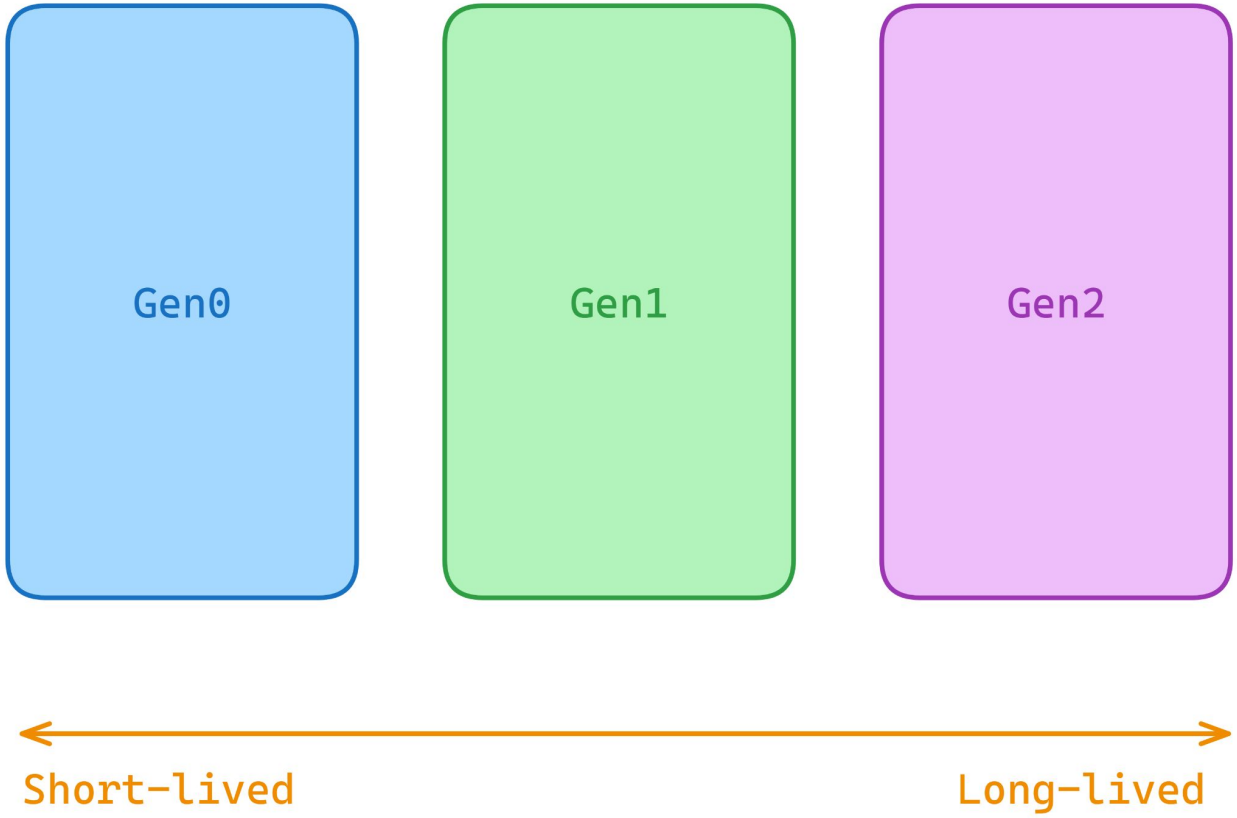
GARBAGE COLLECTOR: DESALOCAÇÃO



GARBAGE COLLECTOR: COMPACTAÇÃO



GARBAGE COLLECTOR: GENERATIONS



GARBAGE COLLECTOR: GEN0

INICIA COM TAMANHO DE 256KB - 4MB, PODENDO **AUMENTAR** DE ACORDO COM A NECESSIDADE

OBJETOS QUE **NÃO** FORAM **DESCARTADOS** NESSA GERAÇÃO SÃO **PROMOVIDOS** PARA A **GEN1**

GC ATUA COM **MAIS FREQUÊNCIA** NESSA GERAÇÃO

ARMAZENA OBJETOS RECÉM CRIADOS OU COM TEMPO DE VIDA CURTO (**VARIÁVEIS TEMPORÁRIAS**)

GARBAGE COLLECTOR: GEN1

ARMAZENA OBJETOS QUE “SOBREVIVERAM” NA GEN0.

INICIA COM TAMANHO DE 512KB - 4MB, PODENDO AUMENTAR DE ACORDO COM A NECESSIDADE.

A **FREQUÊNCIA DE ATUAÇÃO** DO GC É **MENOR** NESSA ETAPA (**UMA VEZ QUE OS OBJETOS DESSA GERAÇÃO TEM UM TEMPO DE VIDA MAIOR**).

SERVE COMO “**PONTE**” ENTRE A **GEN0 E GEN2**.

GARBAGE COLLECTOR: GEN2

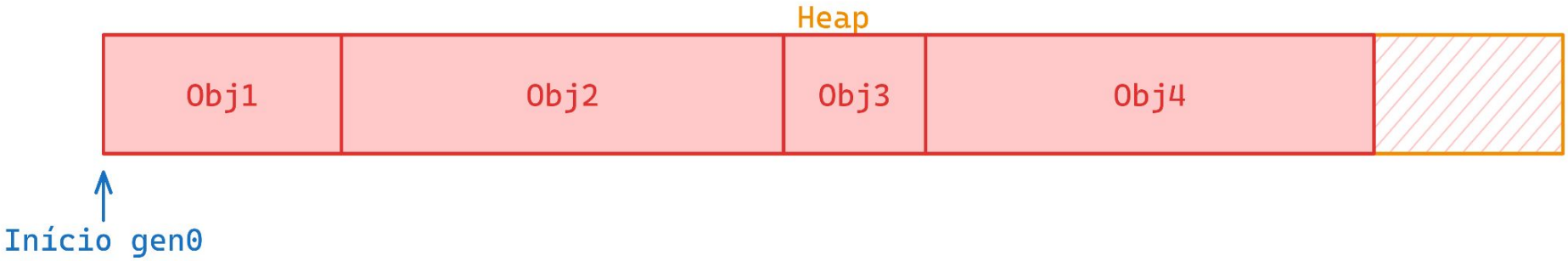
ARMAZENA OBJETOS QUE “**SOBREVIVERAM**” NA **GEN1**.

NÃO POSSUI LIMITE DE TAMANHO, PODENDO SER **ESTENDIDA** ATÉ **2GB** EM SISTEMAS **32BITS** E **8TB** PARA SISTEMAS **64BITS**.

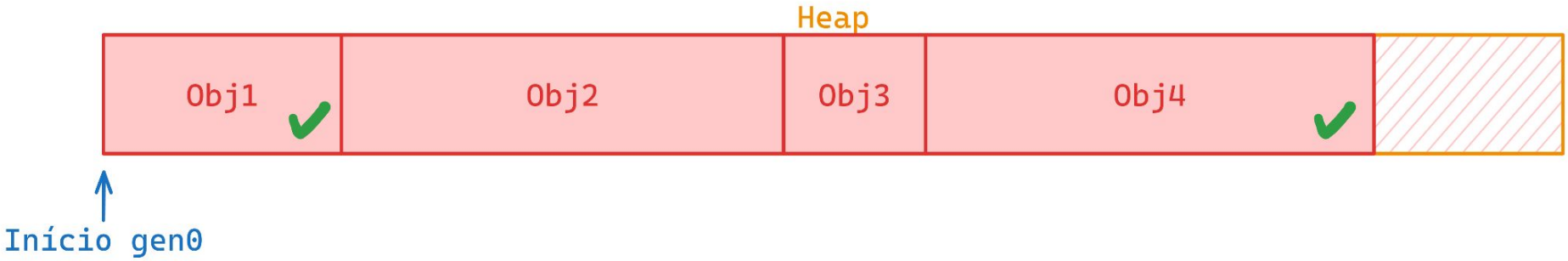
OBJETOS **PERMANECEM** NESSA GERAÇÃO.

ARMAZENA OBJETOS COM UMA LONGA DURAÇÃO DE VIDA.

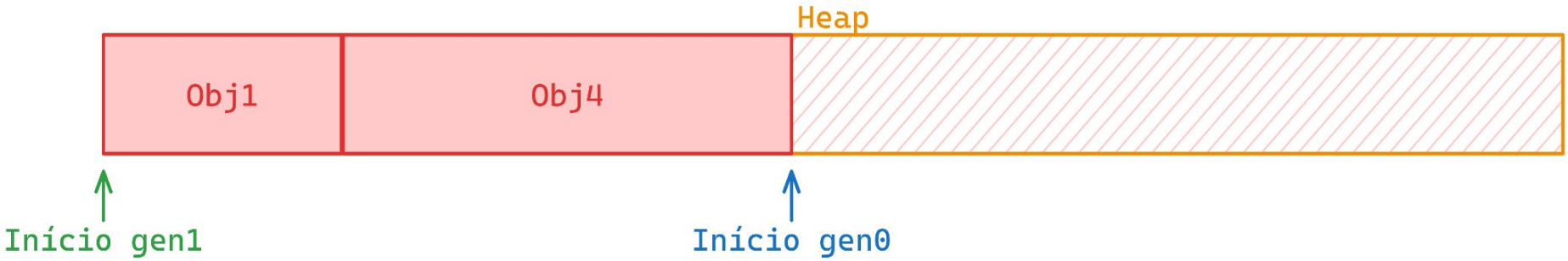
GARBAGE COLLECTOR: FUNCIONAMENTO



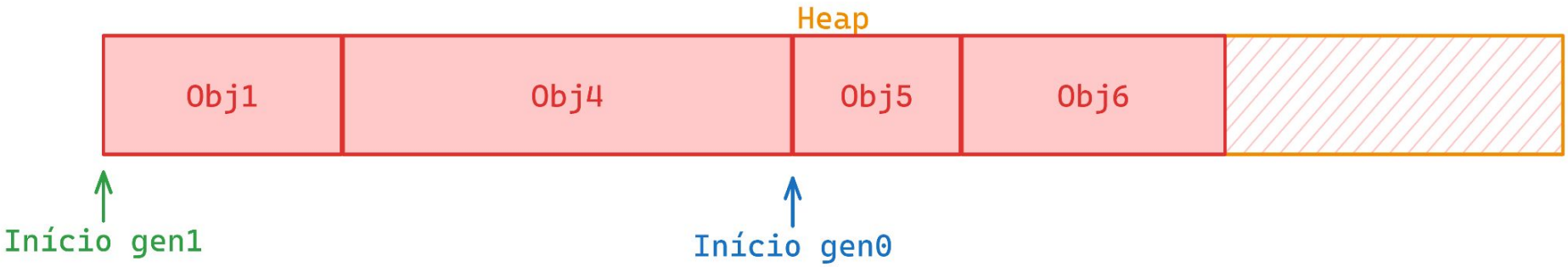
GARBAGE COLLECTOR: FUNCIONAMENTO



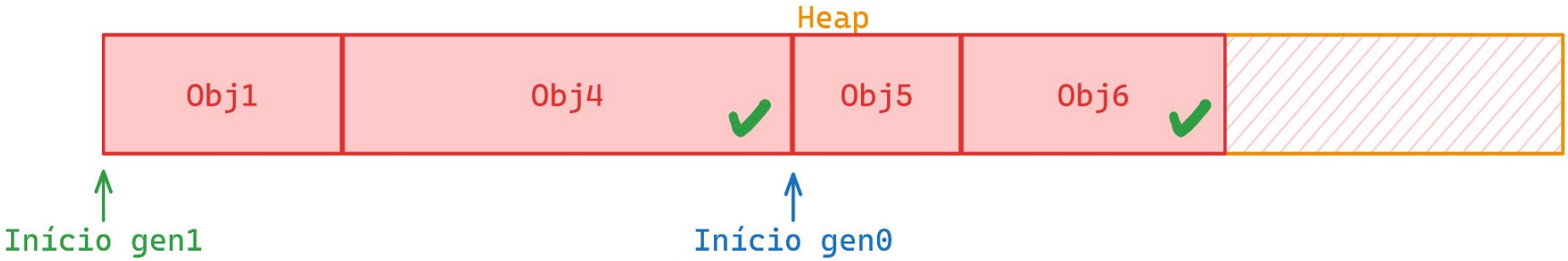
GARBAGE COLLECTOR: FUNCIONAMENTO



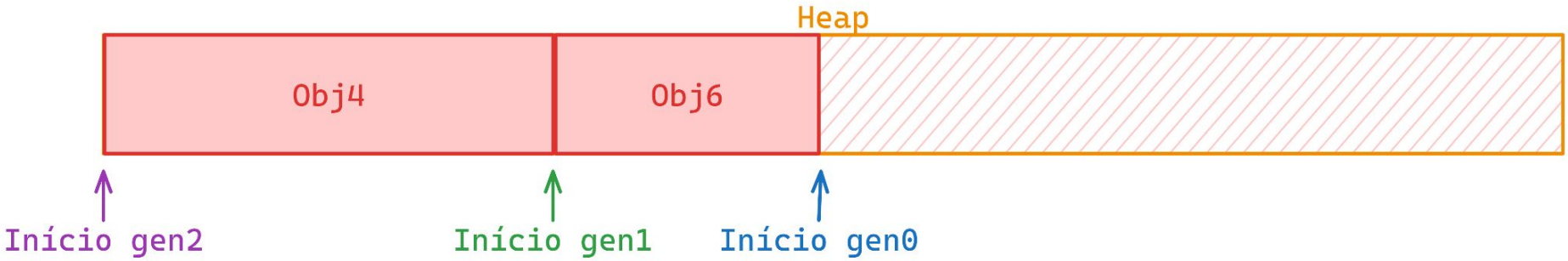
GARBAGE COLLECTOR: FUNCIONAMENTO



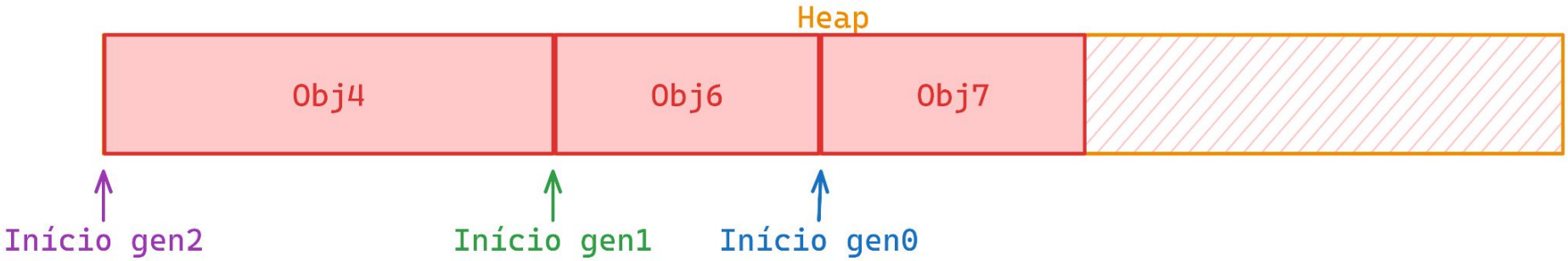
GARBAGE COLLECTOR: FUNCIONAMENTO



GARBAGE COLLECTOR: FUNCIONAMENTO

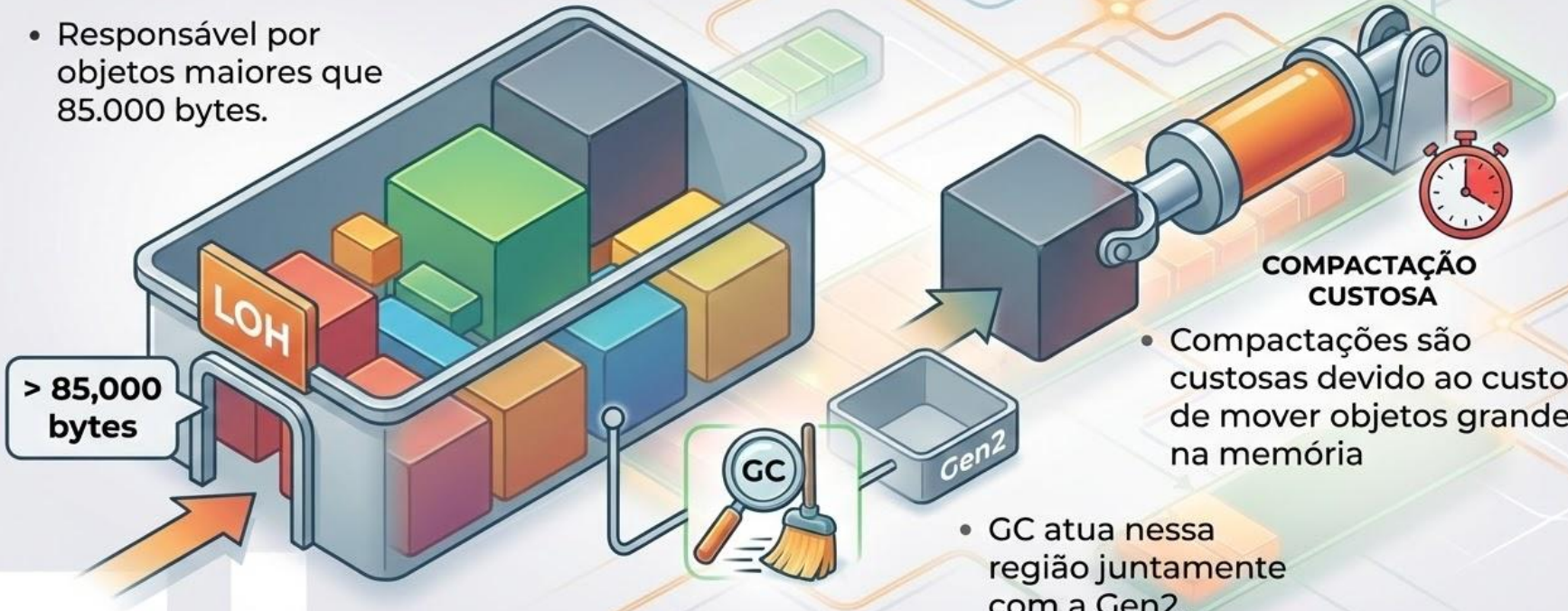


GARBAGE COLLECTOR: FUNCIONAMENTO



GARBAGE COLLECTOR - LOH - Large object heap

- Responsável por objetos maiores que 85.000 bytes.

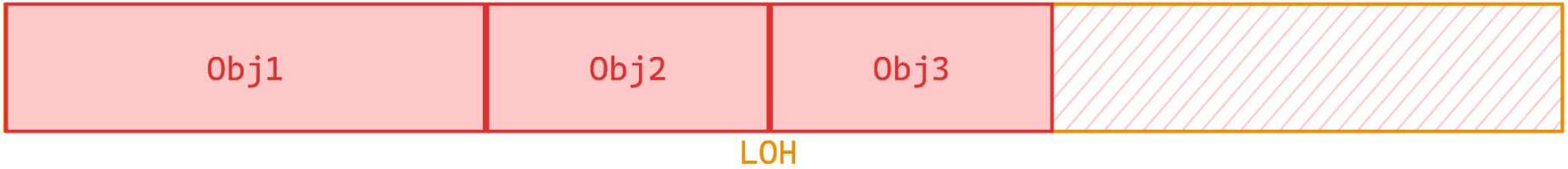


COMPACTAÇÃO CUSTOSA

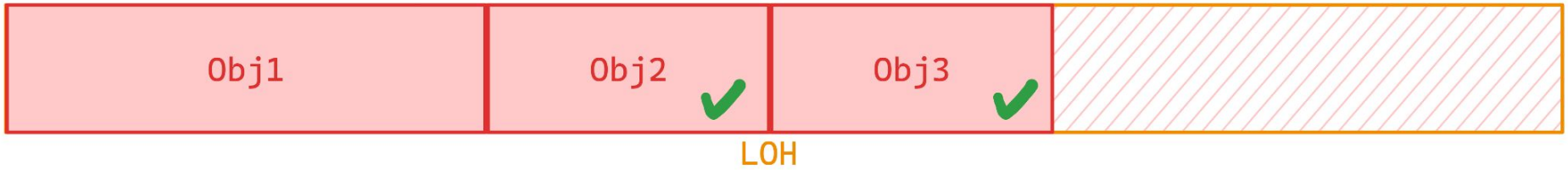
- Compactações são custosas devido ao custo de mover objetos grandes na memória

- GC atua nessa região juntamente com a Gen2.

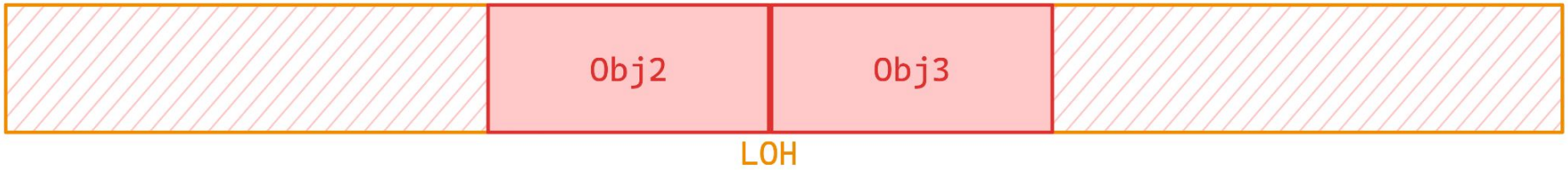
GARBAGE COLLECTOR: LOH



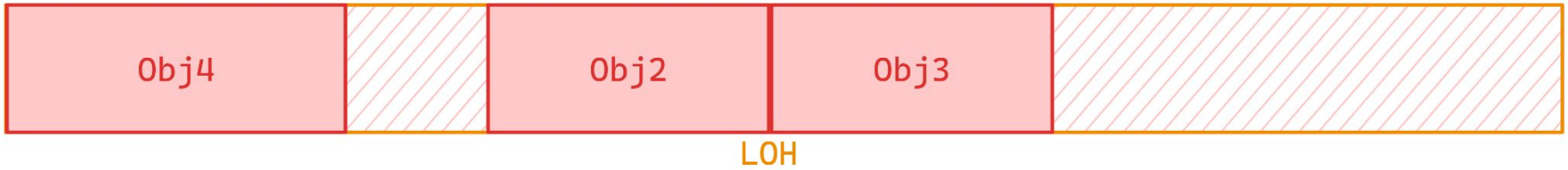
GARBAGE COLLECTOR: LOH



GARBAGE COLLECTOR: LOH



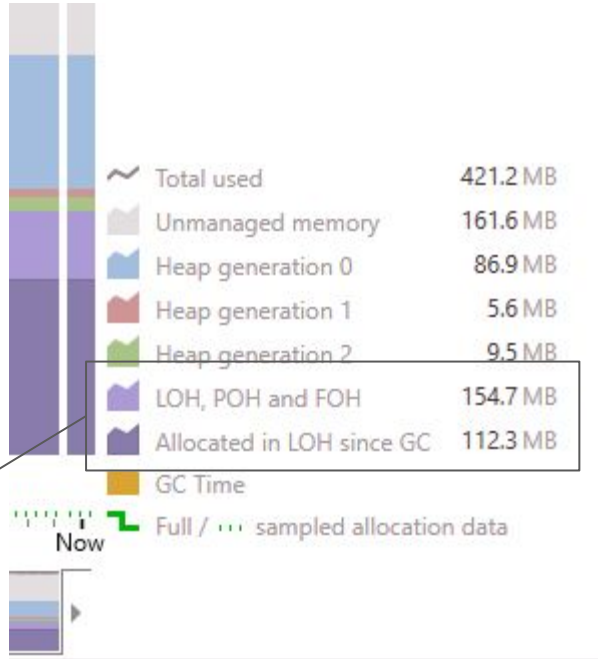
GARBAGE COLLECTOR: LOH



GARBAGE COLLECTOR: LOH

Qual insight podemos
ter a partir dessa alocação?

Alto volume de alocação na LOH



CONSIDERAÇÕES GARBAGE COLLECTOR:

NÃO CHAME **GC.COLLECT()**. SE VOCÊ ACHA QUE PRECISA, O PROBLEMA QUASE SEMPRE É **ALOCAÇÃO DEMAIS**.

FUJA DO LOH: OBJETOS > 85.000 BYTES **CUSTAM CARO**.

REDUZA ALOCAÇÕES: PRIORIZE ESTRUTURAS QUE NÃO VÃO PARA A **HEAP** QUANDO FIZER SENTIDO.

GC É RÁPIDO — ATÉ VIRAR RUÍDO: MILHARES DE COLETAS EM POUCO TEMPO **VIRAM LATÊNCIA E PERDA DE THROUGHPUT**.

CAPÍTULO 4

STRINGS

CONSIDERAÇÕES STRINGS:

STRINGS SÃO **IMUTÁVEIS.**

MUTAÇÕES CRIAM **NOVAS STRINGS**, ALOCANDO **MAIS ESPAÇO** NA **HEAP.**

STRINGS SÃO UM DOS **PRINCIPAIS** MOTIVOS DE **MEMORY LEAK**.

DICAS E BOAS PRÁTICAS SOBRE STRINGS:

UTILIZE O **STRINGBUILDER** PARA MANIPULAR STRINGS.

UTILIZE O **READONLYSPAN** PARA OPERAÇÕES DE CONSULTA.

EVITE UTILIZAR O **SUBSTRING** DE FORMA **EXCESSIVA**.

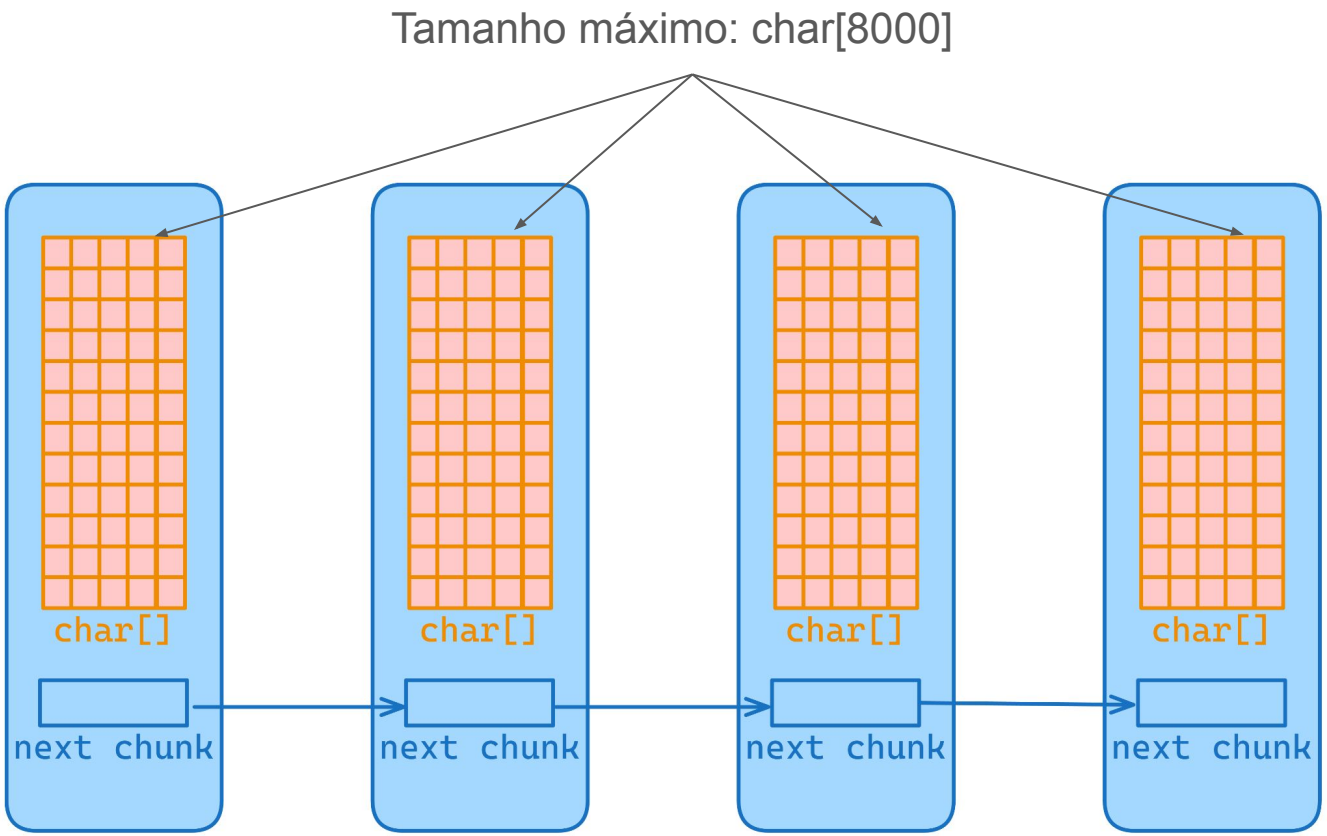
UTILIZE O **STRING.FORMAT** OU **STRINGBUILDER** PARA CONCATENAR LISTAS DE STRINGS

```
foreach (var number in Enumerable.Range(0, 1000))
{
    finalString += number;
}

foreach (var number in Enumerable.Range(0, 1000))
{
    _sb.Append(number);
}
```

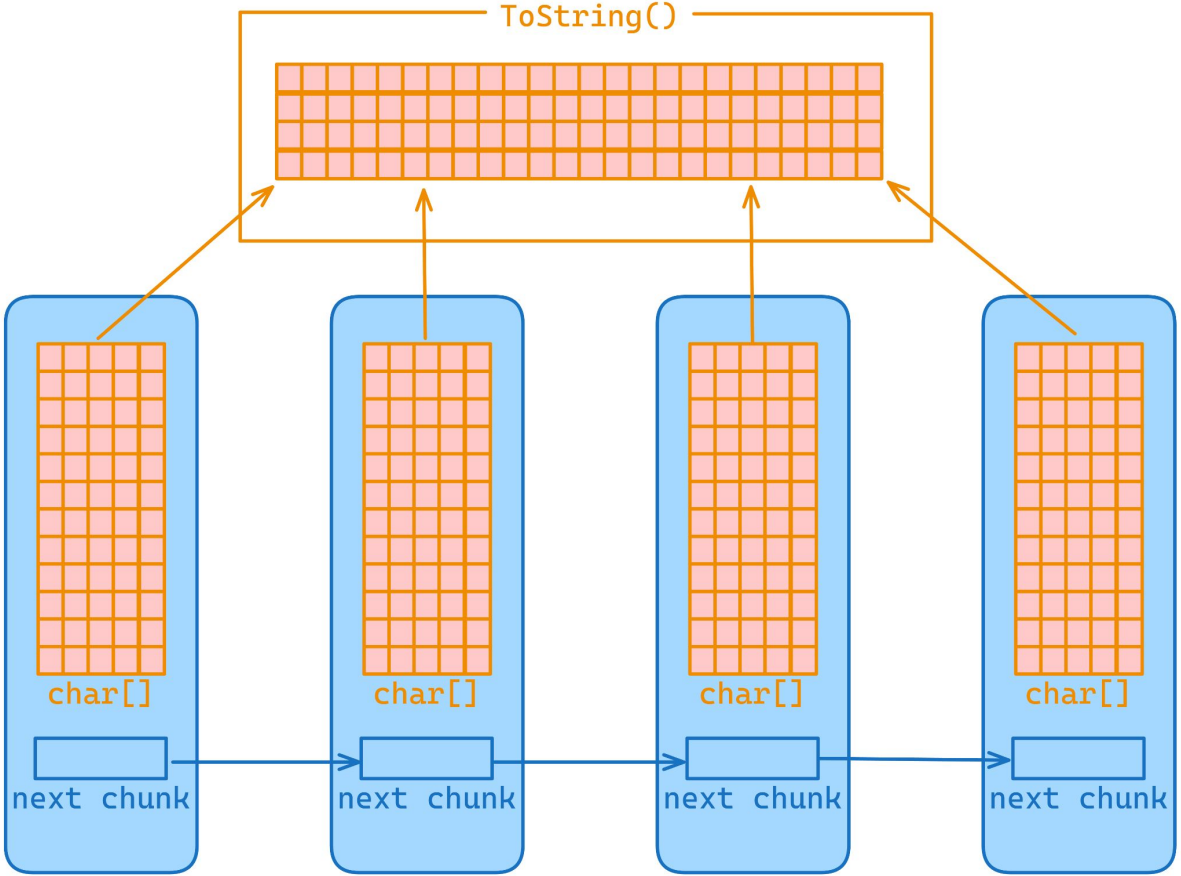
Method	Mean	Error	StdDev	Gen0	Gen1	Allocated
Concat	100.933 us	1.9836 us	4.5973 us	169.7998	2.4414	2773.92 KB
StringBuilder	3.367 us	0.0660 us	0.0734 us	0.3471	-	5.71 KB

COMO O STRINGBUILDER ALOCA TÃO POUCA MEMÓRIA?

REPRESENTAÇÃO INTERNA



REPRESENTAÇÃO INTERNA



READONLYSPAN VS SUBSTRING

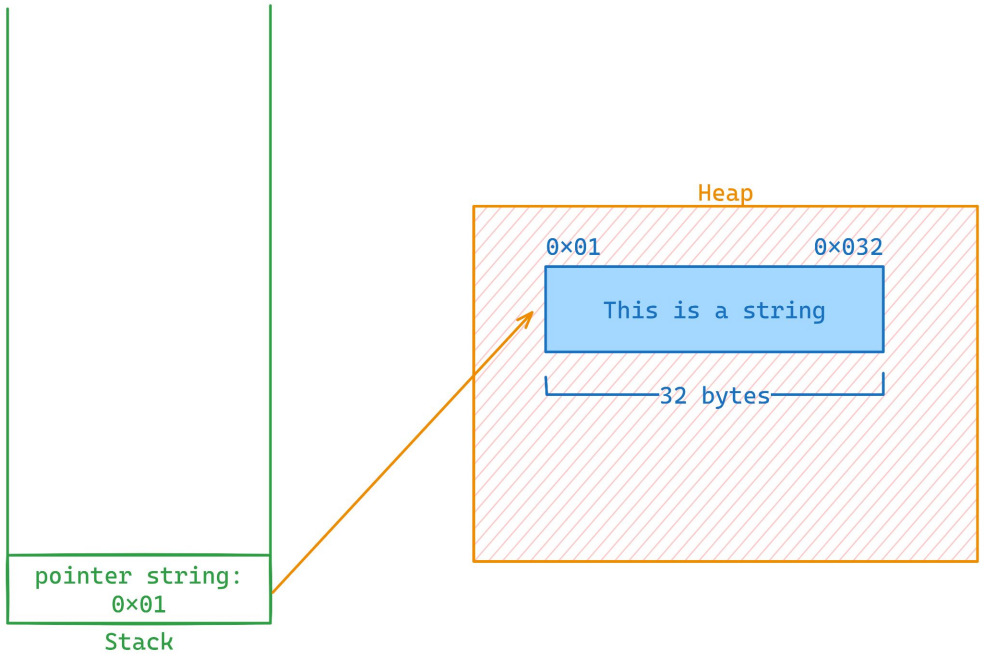

```
var rating = "2.5,1,31,1260759144";
var substring = decimal.Parse(_rating.Substring(0, 3));
var readonlySpan = decimal.Parse(_rating.AsSpan().Slice(0, 3));
```

Method	Mean	Error	StdDev	Gen0	Allocated
Substring	34.14 ns	0.694 ns	1.401 ns	0.0019	32 B
ReadonlySpan	30.54 ns	0.619 ns	0.760 ns	-	-

COMO O READONLYSPAN NÃO ALOCA MEMÓRIA NA HEAP?

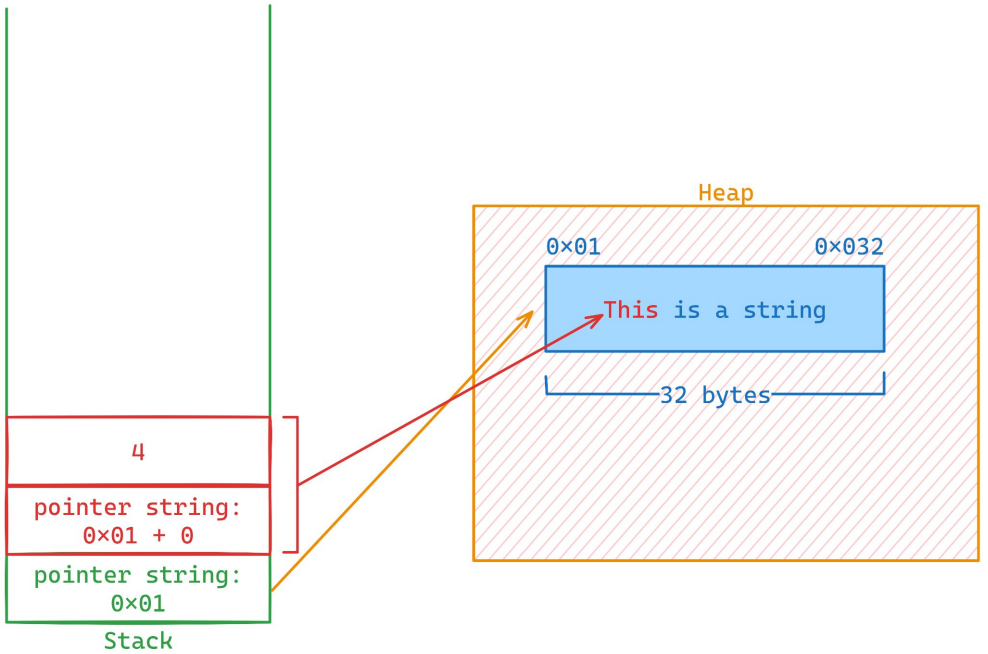
READONLYSPAN


```
var text = "This is a string";
var part1 = text.AsSpan().Slice(0, 4);
var part2 = text.AsSpan().Slice(5, 2);
var part3 = text.AsSpan().Slice(8, 1);
var part4 = text.AsSpan().Slice(10, 6);
```



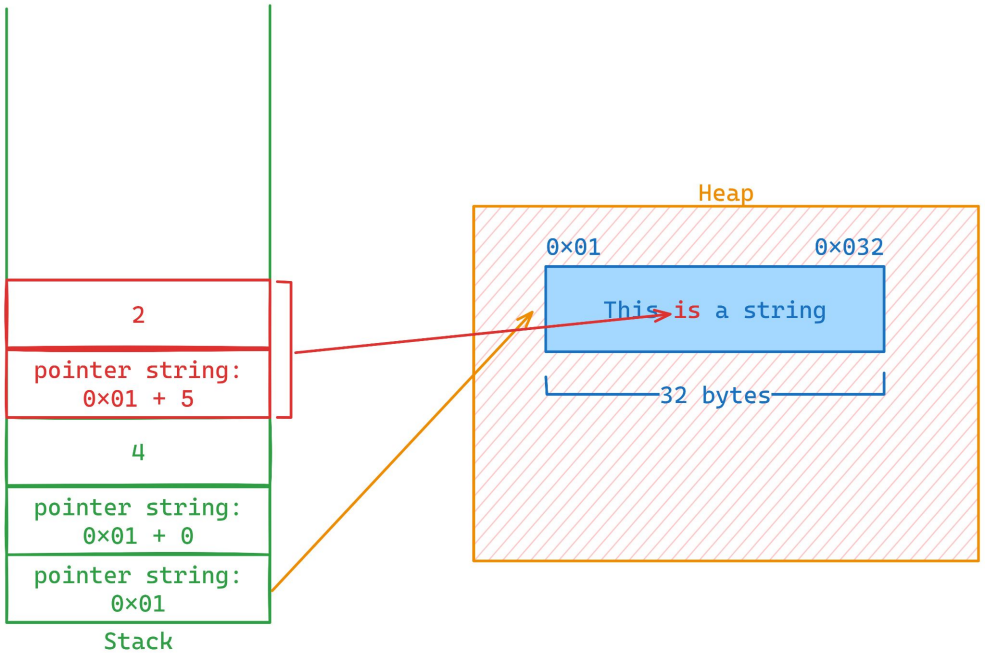
READONLYSPAN


```
var text = "This is a string";
var part1 = text.AsSpan().Slice(0, 4);
var part2 = text.AsSpan().Slice(5, 2);
var part3 = text.AsSpan().Slice(8, 1);
var part4 = text.AsSpan().Slice(10, 6);
```



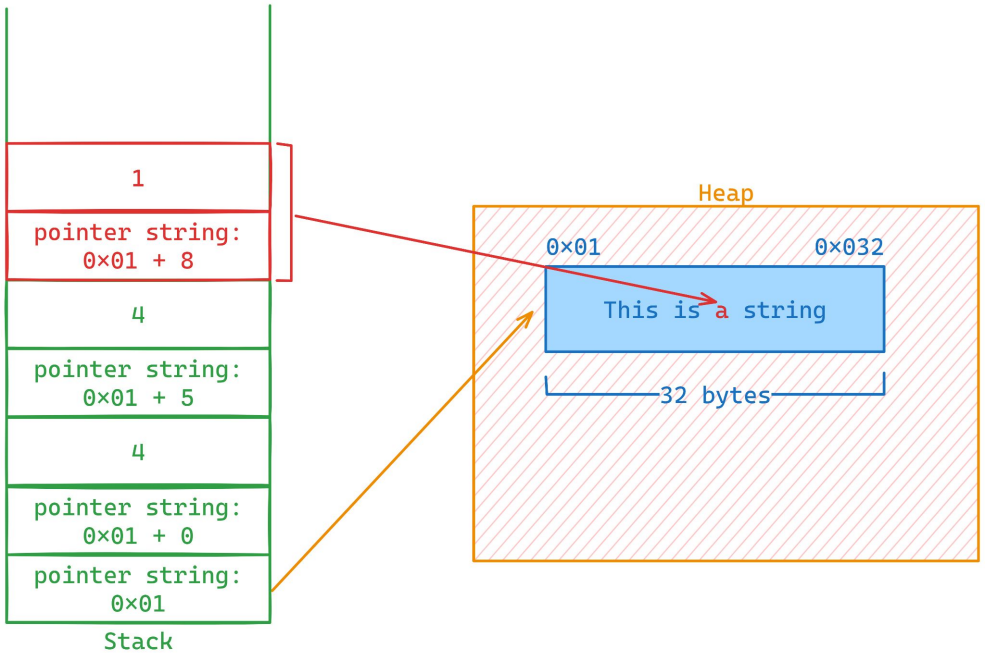
READONLYSPAN


```
var text = "This is a string";
var part1 = text.AsSpan().Slice(0, 4);
var part2 = text.AsSpan().Slice(5, 2);
var part3 = text.AsSpan().Slice(8, 1);
var part4 = text.AsSpan().Slice(10, 6);
```



READONLYSPAN

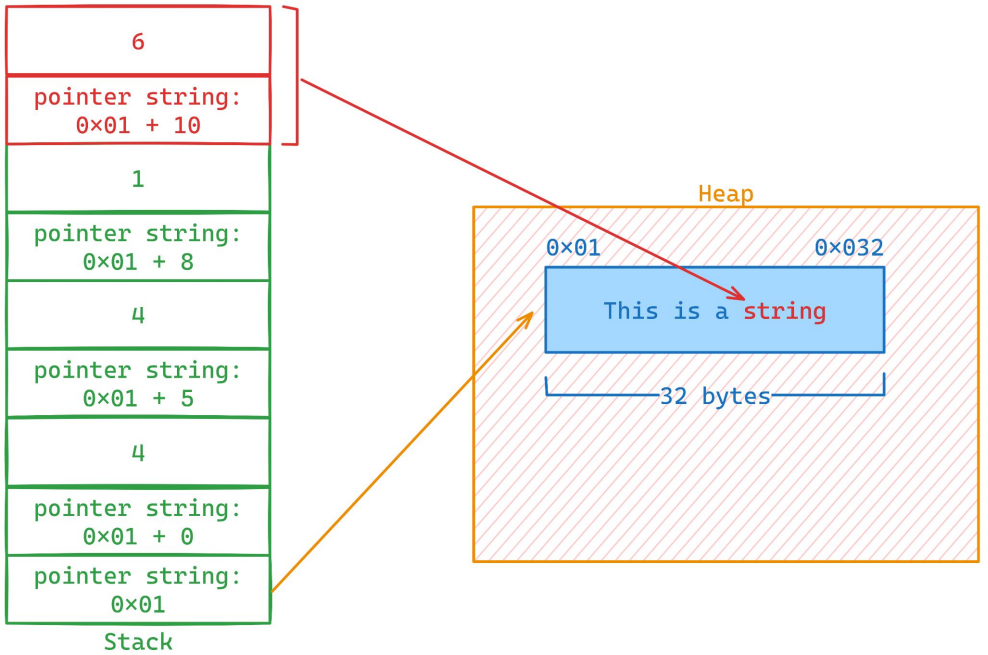

```
var text = "This is a string";
var part1 = text.AsSpan().Slice(0, 4);
var part2 = text.AsSpan().Slice(5, 2);
var part3 = text.AsSpan().Slice(8, 1);
var part4 = text.AsSpan().Slice(10, 6);
```



READONLYSPAN

CLUBE DE ESTUDOS
BY ELEMAR JR

```
var text = "This is a string";
var part1 = text.AsSpan().Slice(0, 4);
var part2 = text.AsSpan().Slice(5, 2);
var part3 = text.AsSpan().Slice(8, 1);
var part4 = text.AsSpan().Slice(10, 6);
```



CAPÍTULO 5

CLASSES, STRUCTS E RECORDS

DEFINIÇÕES CLASSES:

COMPARAÇÃO POR **REFERÊNCIA**

OBJETOS COMPLEXOS **MUTÁVEIS** (ENTIDADES/AGREGADOS)

ALOCADAS NA **HEAP**

DEFINIÇÕES STRUCTS:

NÃO SUPORTAM **HERANÇA**

COMPARAÇÃO POR **VALOR**

OBJETOS LEVES (**IMUTÁVEIS**) - VALUE OBJECTS (DDD)

ALOCADA NA **STACK**

DEFINIÇÕES RECORDS:

ALOCADO NA **HEAP**

IMUTÁVEIS POR PADRÃO

COMPARAÇÃO POR **VALOR**

REQUEST/RESPONSES/DTOS.

PODEM SER ARMAZENADOS NA **STACK** (RECORD STRUCT)

CAPÍTULO 5

TIPOS DE LISTAS

ARRAYS: DEFINIÇÕES

TAMANHO **FIXO**

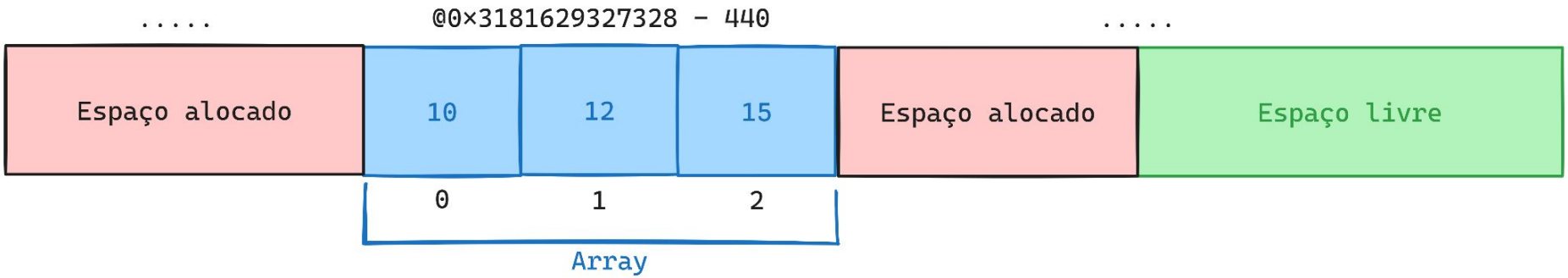
ALOCA MEMÓRIA DE FORMA **CONTÍGUA**

IMUTÁVEL

ARRAY ALOCAÇÃO

CLUBE DE ESTUDOS
BY ELEMAR JR

```
var array = new int[] { 10, 12, 15 };
```



CARACTERÍSTICAS DA LIST:

TAMANHO **DINÂMICO**

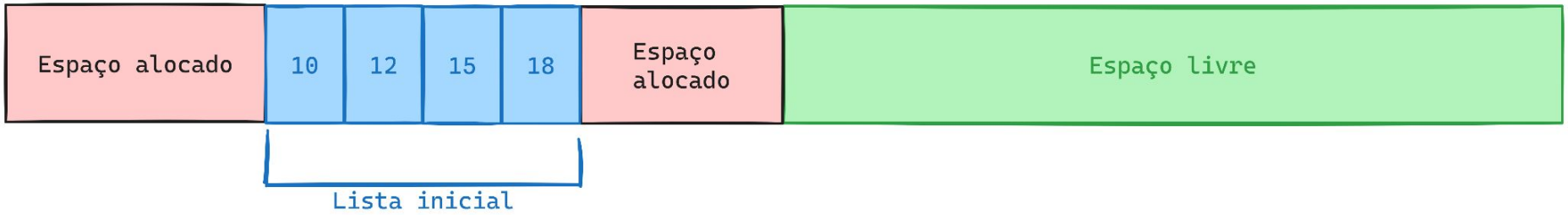
IDEAL PARA CENÁRIOS ONDE **NÃO SABEMOS A QUANTIA DE ITENS** QUE SERÃO ADICIONADOS

QUANDO A CAPACIDADE É **EXCEDIDA**, UM NOVO ARRAY COM O **DOBRO DO TAMANHO** É CRIADO E OS ELEMENTOS SÃO **COPIADOS**

LIST - MEMÓRIA

CLUBE DE ESTUDOS
BY ELEMAR JR

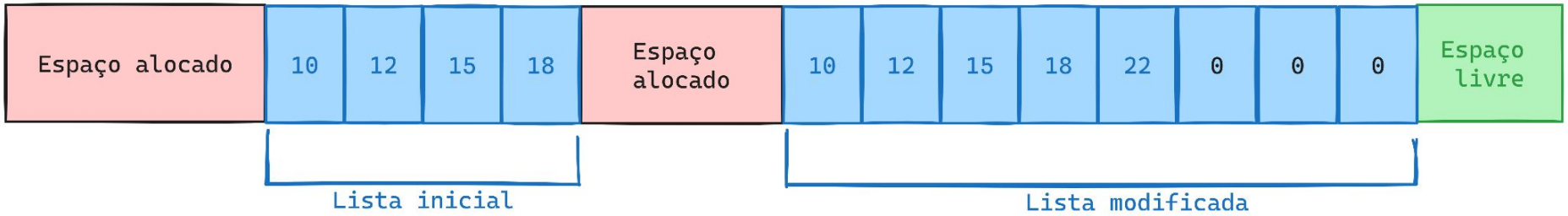
```
var list = new List<int>()
{
    10, 12, 15, 18
};
```



LIST - MEMÓRIA

CLUBE DE ESTUDOS
BY ELEMAR JR

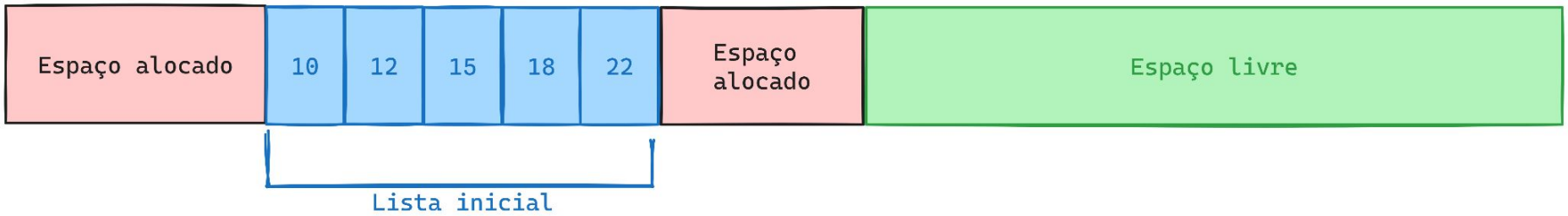
```
var list = new List<int>()
{
    10, 12, 15, 18
};

list.Add(item:22);
```



LIST - MEMÓRIA

CLUBE DE ESTUDOS
BY ELEMAR JR

```
var list = new List<int>(capacity:5)
{
    10, 12, 15, 18, 22
};
```



LIST: DICAS E BOAS PRÁTICAS

SEMPRE USE A PROPRIEDADE **COUNT** OU **LENGTH**

EVITE **RETORNAR/RECEBER** LIST NOS MÉTODOS

SEMPRE QUE POSSÍVEL, **LIMITE O TAMANHO DA LISTA**

EVITE UTILIZAR O **.TOLIST()** ONDE **NÃO** FOR NECESSÁRIO

LIST - BENCHMARK LISTA TAMANHO FIXO

CLUBE DE ESTUDOS
BY ELEMAR JR

Method	N	Mean	Error	StdDev	Gen0	Allocated
SizeFixed	10	21.68 ns	0.468 ns	0.415 ns	0.0057	96 B
NoSizeFixed	10	50.00 ns	1.037 ns	1.110 ns	0.0129	216 B
SizeFixed	100	82.98 ns	1.701 ns	2.440 ns	0.0272	456 B
NoSizeFixed	100	226.58 ns	4.547 ns	4.466 ns	0.0706	1184 B
SizeFixed	1000	738.81 ns	14.635 ns	13.689 ns	0.2422	4056 B
NoSizeFixed	1000	2,358.39 ns	46.221 ns	56.764 ns	0.5035	8424 B

LIST - BENCHMARK COUNT VS COUNT()

CLUBE DE ESTUDOS
BY ELEMAR JR

Method	N	Mean	Error	StdDev	Median	Allocated
CountProperty	10	0.0624 ns	0.0249 ns	0.0732 ns	0.0010 ns	-
CountMethod	10	1.7103 ns	0.0448 ns	0.0419 ns	1.7060 ns	-
CountProperty	100	0.0026 ns	0.0033 ns	0.0031 ns	0.0019 ns	-
CountMethod	100	1.7167 ns	0.0560 ns	0.0523 ns	1.6829 ns	-
CountProperty	1000	0.0089 ns	0.0085 ns	0.0080 ns	0.0101 ns	-
CountMethod	1000	1.7179 ns	0.0497 ns	0.0465 ns	1.7065 ns	-

CARACTERÍSTICAS HASHSET:

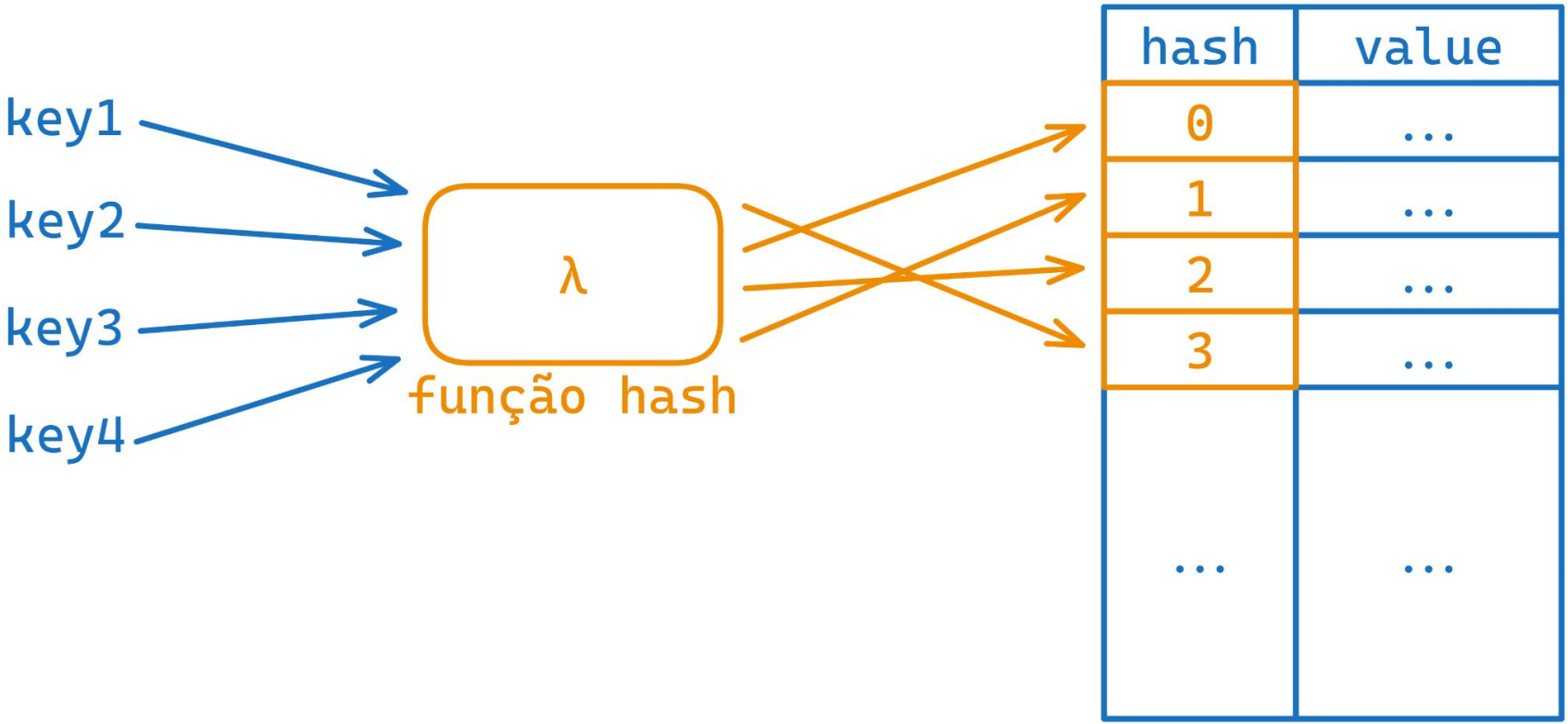
GARANTE A **INEXISTÊNCIA** DE **ELEMENTOS DUPLICADOS**

IMPLEMENTADO UTILIZANDO **HASHTABLE**

IDEAL PARA **CENÁRIOS** ONDE **INSERÇÕES E BUSCAS** PRECISAM SER FEITAS DE **FORMA RÁPIDA**

MÉTODO **GETHASHCODE** DOS ELEMENTOS É UTILIZADO EM **BUSCAS**, **INSERÇÕES** E **REMOÇÕES**

HASHTABLE



COMPARAÇÃO DE BUSCA EM LIST E HASHTABLE


```
public class BenchmarkPlayground
{
    readonly List<int> _list = [];
    readonly HashSet<int> _hashSet = [];

    [Params(1000, 10_000, 1_000_000)]
    public int N;

    [GlobalSetup]
    public void Start()
    {
        for (int i = 0; i <= N; i++)
        {
            _list.Add(i);
            _hashSet.Add(i);
        }
    }

    [Benchmark]
    public bool List() => _list.Contains(N);

    [Benchmark]
    public bool Hashset() => _hashSet.Contains(N)
}
```

COMPARAÇÃO DE BUSCA EM LIST E HASHTABLE

CLUBE DE ESTUDOS
BY ELEMAR JR

Method	N	Mean	Error	StdDev	Allocated
List	1000	41.289 ns	19.330 ns	1.0595 ns	-
Hashset	1000	2.970 ns	1.457 ns	0.0799 ns	-
List	10000	564.794 ns	660.389 ns	36.1981 ns	-
Hashset	10000	2.687 ns	1.725 ns	0.0945 ns	-
List	1000000	57,226.538 ns	26,254.094 ns	1,439.0755 ns	-
Hashset	1000000	2.720 ns	1.440 ns	0.0789 ns	-

COMPARAÇÃO DE PERFORMANCE

Collection	Tempo inserção	Tempo remoção	Tempo busca
List<T>	O(1)*	O(n)	O(n)
LinkedList<T>	O(1)	O(1)	O(n)
Dictionary<K,V>	O(1)	O(1)	O(1)
HashSet<T>	O(1)	O(1)	O(1)
Queue<T>	O(1)*	O(1)	-
Stack<T>	O(1)*	O(1)	-

* Algumas inserções podem demorar O(n), mas a maioria leva O(1)